Text-to-Constraints: Extracting Data Quality Constraints from Regulatory Text

Eduardo H. M. Pena
Federal University of Technology-Paraná
Brazil, Campo Mourão
eduardo@utfpr.br

Junio de Freitas
Federal University of Amazonas
Brazil, Manaus
junio.freitas@icomp.ufam.edu.br

Eduardo Cunha de Almeida
Federal University of Paraná
Brazil, Curitiba
eduardo@ufpr.br

Altigran da Silva
Federal University of Amazonas
Brazil, Manaus
alti@icomp.ufam.edu.br

Abstract

Many regulatory documents specify requirements that valid data must satisfy, but those specifications are rarely available as data quality constraints. Data profiling, particularly constraint discovery, is a well-studied alternative when constraints are unavailable, but discovered constraints reflect what the data satisfies rather than what it is supposed to satisfy. We introduce the *Text-to-Constraints* problem: given a dataset and related regulatory text, extract executable data quality constraints grounded in that text. Then, we present the first framework to address this problem, `Norma`, which identifies verifiable requirements in regulatory documents and maps them to denial constraints using a rich schema-grounded predicate vocabulary. We evaluate `Norma` on real-world datasets with diverse textual and data characteristics, covering constraints ranging from simple checks to complex cross-record rules. Across constraint classification, extraction, and violation detection, `Norma` outperforms baselines from data profiling and Text-to-SQL, recovering constraints they miss and identifying real regulatory violations with high precision in publicly released data.

PVLDB Reference Format:
Eduardo H. M. Pena, Junio de Freitas, Eduardo Cunha de Almeida, and Altigran da Silva. Text-to-Constraints: Extracting Data Quality Constraints from Regulatory Text. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/EduardoPena/norma-text-to-constraints.

1 Introduction

Many data management tasks rely on *data quality constraints*, which define what constitutes valid data. However, such constraints are often missing, incomplete, or outdated in real-world datasets. This gap has motivated a large body of work on *data profiling*, where data constraints are automatically discovered from the data itself [2].

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Profiling algorithms extract constraints, such as functional dependencies (FDs) and denial constraints (DCs), by identifying patterns that hold over a given data instance, and have been widely used to support data cleaning and error detection [57].

However, a fundamental assumption underlying this line of work is that constraints must be inferred from data, and thus, they are limited to discovering constraints that reflect what the data *happens to satisfy*, not what it is *required to satisfy*, given some policy, regulation, or legislation. No profiling algorithm can make that distinction, nor can it surface constraints that must hold by design but are violated in the data.

ALTI: [R3-O1] Suavizar esta afirmação: ela foi lida como factualmente incorreta. (i) Existe muito trabalho guiando especialistas a definir/refinar regras, e (ii) a descoberta *aproximada* de DCs ("Discovery of Approximate (and Exact) Denial Constraints", VLDB 2019) já descobre regras que NÃO valem plenamente nos dados. Reescrever para distinguir "regras requeridas por norma" de "regras que valem (exata ou aproximadamente) nos dados", e citar a descoberta aproximada de DCs aqui.

Many datasets are governed by rules specified in regulatory documents, policy manuals, and operational specifications, which define requirements such as mandatory fields, valid values, eligibility criteria, uniqueness conditions, quotas, and ordering rules [7, 12, 55]. Formalizing these rules as constraints is difficult because they depend on combinations of attributes, relationships across tuples, comparison operators, thresholds, and context-dependent conditions, requiring expressive *data quality constraints*. However, since such rules are typically expressed only in natural language, they are rarely translated into executable constraints and used with the data they govern. Consequently, datasets are frequently released without an explicit representation of their own correctness requirements, so violations remain undetected and subsequent analyses may be based on data that does not satisfy their intended specification.

This problem is common in modern data settings, where open-data portals publish standalone files with limited metadata, data lakes aggregate heterogeneous sources, and data pipelines connect systems whose semantics are only partially documented [14, 35].

Example 1 (Brazilian public university admission). *Brazil's "Sistema de Seleção Unificada" (SISU) manages the public university placements for millions of students annually. The application data*

is publicly available,[1] *with sensitive fields anonymized. The dataset schema includes attributes such as* student_id, course_code, shift, institution_code, year, *and* edition. *The rules that define correct enrollment outcomes, however, are described in separate documents that do not accompany the dataset. For instance, Normative Ruling No. 21/2012*[2] *is a PDF file found on the Brazilian government website that states (Article 19):*[3]

> *"... the student will be ranked in descending order of scores for the seat option they applied to, respecting the available seats at the institution, by campus location, course, and shift, as well as the student's admission category.."*

This passage defines an ordering constraint*: within each context defined by institution, campus, course, shift, and admission category (and per year/edition), a higher score should not result in a worse rank. The following SQL query checks for violations of this requirement in SISU's application data:*

```
SELECT t.student_id, t'.student_id
  FROM applications t, applications t'
 WHERE t.student_id <> t'.student_id
   -- same context
   AND t.year = t'.year
   AND t.edition = t'.edition
   AND t.institution_id = t'.institution_id
   AND t.campus_id = t'.campus_id
   AND t.course_code = t'.course_code
   AND t.shift = t'.shift
   AND t.application_mode = t'.application_mode
   -- violation: higher score but worse ranking
   AND t.score > t'.score
   AND t.ranking > t'.ranking;
```

Any nonempty result set for this query indicates a violation of a legal requirement.

ALTI: [R2-O1] Reportar AQUI o número concreto de tuplas do SISU que violam o Artigo 19 (esta query). Toda a motivação assume que datasets publicados violam silenciosamente suas normas; dar o número empírico do exemplo motivador torna a motivação crível. O número existe na Tabela 2 (487 violações no total); extrair o valor específico desta regra.

□

In this paper, we introduce the *Text-to-Constraints* problem: given a dataset and a set of normative requirements derived from related regulatory text, ~~extract data-quality constraints grounded in that text to validate the data~~→map those requirements to executable data-quality constraints over the dataset schema.

[COMMENT: ALTI, 2026-08-26: Reframed the problem around grounding given normative requirements; extraction of requirements from regulatory text is outside the current scope.]

This problem differs from traditional constraint discovery in that, instead of inferring constraints from patterns observed in the data instance, which may reflect incidental regularities rather than intended rules, it requires reasoning over regulatory semantics, schema structure, and violation logic under a chosen constraint formalism. The *Text-to-Constraints* problem is related to the well-known Text-to-SQL [56] problem as both involve mapping natural-language specifications to structured representations grounded in a database schema. However, Text-to-SQL generates queries to answer specific user questions, whereas Text-to-Constraints ~~extracts declarative invariants~~→maps normative requirements to executable constraints that must hold for all valid dataset instances.

[COMMENT: ALTI, 2026-08-26: Aligned the Text-to-SQL comparison with the revised problem definition.]

Challenges. ~~Regulatory documents are typically long and heterogeneous, and only a fraction of their sentences carry normative force [17] (i.e., sentences that prescribe what the data is required to satisfy). In SISU, for example, the normative text spans several pages across multiple articles, yet only a handful of passages describe conditions that can be evaluated against the released data. The remaining passages consist of section titles, definitions, procedural guidance, rationale, and contextual descriptions, which are essential for human readers but do not directly imply data validity. Rules are interleaved throughout this material, often qualified by scope conditions, numeric thresholds, or exception clauses, and expressed in the varied, imprecise language that characterizes legal and administrative text. Translating such passages into formal constraints requires identifying which sentences carry normative force by resolving implicit references to schema elements. Regulatory documents are formed by natural language sentences that carry normative force~~→Regulatory documents consist of natural-language text, including sentences that carry normative force [17] (i.e., sentences that prescribe what the data is required to satisfy). Thus, the first challenge we address is mapping these natural-language rules that define valid data to executable data-quality constraints. Example 1 illustrates this challenge: Article 19 of the SISU regulation states an ordering requirement in prose, while checking it requires making explicit the relevant context, attributes, tuple comparisons, and violation condition. More generally, a normative passage may combine conditions, qualifiers, cross-record relationships, numeric thresholds, and exceptions, while leaving implicit how its terms correspond to elements of the dataset schema. Mapping such a passage to an executable constraint therefore requires resolving its semantics, identifying the relevant schema elements, and specifying what constitutes a violation.

[COMMENT: ALTI, 2026-08-26: Reframed the first challenge around the text-to-constraint task and retained SISU as a concrete example, without claiming to identify normative passages in an entire regulatory document.]

A second challenge is determining the correct scope of each constraint. Omitting qualifiers results in overly broad constraints, flagging valid data as violations. Adding conditions to the rule results in constraints that are too narrow, missing violations. In Example 1, omitting qualifiers such as year/edition or admission category mis-scopes the check and can flag valid outcomes. On the other hand, if one treats "respecting the limit of available seats" as `AND t.ranking <= seats AND t'.ranking <= seats` and

[1] https://dadosabertos.mec.gov.br/sisu
[2] https://sisugestao.mec.gov.br/docs/portaria-2012-21.pdf
[3] Schema elements and quoted statements translated from Portuguese for readability.

only compares applicants within the admitted prefix, then scores involving waitlisted applicants are never checked.

ALTI: [R1-O2, R3-O6] Reescrever este "terceiro desafio": foi lido como confuso por dois revisores. Problema: "SQL é um moving target" não ficou claro, e como toda DC se escreve em SQL, parece contradizer a escolha de DCs como saída. Esclarecer que o ponto NÃO é SQL-vs-não-SQL, mas escolher um espaço-alvo canônico/restrito (DCs) em vez do espaço aberto de formulações SQL equivalentes. Ou seja, a "moving target" é a falta de forma canônica do SQL para extração/comparação. Conectar explicitamente com o argumento "Why DCs" das Preliminaries.

A third challenge is the output formalism, and there is an unavoidable compromise. General SQL is expressive, but it is a moving target for extraction, making it difficult to standardize checks across documents. More restricted languages are easier to capture and evaluate, but they miss many cross-record rules, including ordering, quotas, and conditional uniqueness. Example 1 requires comparing applicants within the same context, so we need a constraint language that supports such comparisons and can still be checked efficiently on large datasets.

~~Generally speaking, the Text-to-Constraints problem can be seen as a form of information extraction, in which systems have traditionally required extensive domain-specific annotation and feature engineering [6].~~ Generally speaking, the Text-to-Constraints problem is to map natural-language regulatory text to executable data-quality constraints grounded in a target dataset schema. This mapping requires linking the terms of a normative passage to schema elements, expressing its semantics in a constraint formalism, and specifying executable violation conditions.

[COMMENT: ALTI, 2026-08-26: Replaced the information-extraction framing with a direct description of the task addressed by Norma.]

~~Large language models have reduced this burden in related settings, such as checking whether software requirements comply with regulations [18].~~ Large language models provide a promising way to perform this mapping from natural-language regulatory text to structured constraints, as demonstrated in related regulatory compliance tasks [18]. But Text-to-Constraints requires more than identifying relevant passages. The output must be linked to the dataset schema, conform to a constraint formalism, and express violation conditions that can be executed against data. None of these requirements follows automatically from language model outputs. They require explicit mechanisms for schema alignment, predicate grounding, and violation semantics.

ALTI: [R1-O1] Posicionar a intro mais a fundo frente ao estado-da-arte, com citações: (i) regras declarativas tipo REEs (Rule/Entity Enhancing rules) que usam ML-predicates, relacionando com nosso espaço de predicados; (ii) como métodos de descoberta de DC poderiam ser estendidos vs. nossa abordagem baseada em texto. Hoje a intro só contrasta com Text-to-SQL e IE. Mover/citar material do Related Work para cá.

From text to data quality constraints. These challenges show that Text-to-Constraints goes beyond simple text extraction. A useful solution must identify passages with normative requirements, link them to the dataset schema, and translate them into formal constraints with the correct scope and violation semantics.

To address these challenges, we propose Norma, the first framework for Text-to-Constraints. Given a regulatory document and a target dataset, Norma identifies sentences with normative force and maps them to formal constraints over the dataset schema. Norma uses denial constraints to represent constraints, as they provide a striking balance between expressiveness and control. Unlike Text-to-SQL systems, which generate ad hoc SQL queries in response to user questions, Norma derives reusable constraints with well-defined violation semantics. It also differs from traditional data profiling. In data profiling, denial constraints are defined over a narrower set of predicates for tractability. Because Norma operates over text instead of searching for regularities in the data, it can use a richer predicate vocabulary.

ALTI: [R3-O5] Termo "regularities" foi apontado como vago. Trocar por algo preciso, p.ex. "padrões estatísticos que valem na instância de dados" (em contraste com requisitos normativos), deixando claro o contraste dados-vs-texto.

This includes multiple operators, column relationships, null checks, domain checks, and pattern predicates. As a result, Norma can capture a broader range of regulatory rules than traditional profiling methods, while still keeping the output space structured and restricted.

Norma supports multiple extraction and mapping modes with different computational-cost trade-offs, ranging from direct, prompt-based variants to structured, multi-step pipelines. Our experiments show that this framework recovers rules that state-of-the-art data profiling algorithms and Text-to-SQL systems do not capture. Norma identifies real compliance failures in publicly released data using constraints extracted from governing text. A key advantage of this setting is that each detected violation can be traced back to the source passage that justifies the constraint.

A summary of our contributions are the following:

- A formalization of the *Text-to-Constraints* problem;
- Norma, the first framework for this problem, with sentence-level and document-level extraction implementations with different cost-effectiveness trade-offs;
- A set of datasets with annotated constraints, and evaluation measures adequate for the Text-to-Constraint problem.
- An extensive empirical study across multiple models, prompting strategies, and baselines from data profiling and Text-to-SQL, showing that Norma recovers critical constraints missed by data-only profiling algorithms.

The remainder of this paper is organized as follows. Section 2 formalizes the Text-to-Constraints problem. Section 3 introduces Norma, whose classifier and mapper components are detailed in Sections 4 and 5. Section 6 presents our experimental evaluation of Norma against different baselines. Section 7 discusses related work, and Section 8 presents our conclusions.

2 Preliminaries

Problem definition. We are given a relational dataset $\mathcal{D}$ and an associated *normative document*. This textual source could be laws, regulations, policy manuals, or operational specifications. It defines the requirements the data must satisfy, expressed in natural language rather than formal, machine-executable constraints.

We model a normative document as a set $\mathcal{S}$ of *segments*, with each segment $s \in \mathcal{S}$ representing a distinct, self-contained textual unit in the document. The segmentation granularity may vary. A segment can correspond, for example, to a single sentence, multiple sentences, or a larger passage. We consider a document normative if it contains at least one segment that expresses a requirement whose compliance can be assessed based on the data.

A *data quality constraint* is a formal rule that $\mathcal{D}$ must satisfy (we discuss the choice of constraint formalism below). Constraints are evaluated on a target relation $r \in \mathcal{D}$ with schema $R = (A_1, \ldots, A_n)$, where each A_i is an attribute with domain $\mathrm{dom}(A_i)$. If a constraint involves multiple relations, we evaluate it on a derived relation obtained by applying the required joins.

Let Φ denote the space of constraints over R. The *Text-to-Constraints* problem is to extract a set of segment–constraint pairs

$$\Pi = \{(s_1, \varphi_1), \ldots, (s_k, \varphi_k)\} \subseteq \mathcal{S} \times \Phi,$$

where $s_i \in \mathcal{S}$ is a segment and $\varphi_i \in \Phi$ is a constraint whose semantics are grounded in the normative content of s_i. Each pair must satisfy two requirements: *executability*, φ_i can be evaluated on any instance of R, and *traceability*, s_i is identifiable as the textual justification for φ_i.

In general, one segment may yield multiple constraints, and a constraint may depend on multiple segments. In this work, we consider two granularities: sentence-level extraction, where each segment is a single sentence and yields at most one constraint, and document-level extraction, where constraints are derived by jointly reasoning over multiple segments within a normative document.

Denial constraints. We instantiate Φ as the space of *denial constraints* (DCs). A DC specifies that a conjunction of predicates must never be simultaneously true. Formally, given a relation instance r over schema R, a DC φ is a first-order sentence of the form:

$$\varphi : \forall t_1, \ldots, t_m \in r : \neg\big(p_1 \wedge \cdots \wedge p_\ell\big),$$

where each p_i is an atomic predicate over tuple variables and constants, and $m \geq 1$ is the number of tuple variables. While DCs may in principle involve an arbitrary number of tuple variables, we focus on DCs with $m \leq 2$. This restriction is common in the literature and adopted by prior work [15, 39, 46], as DCs involving more than two tuple variables are rare in practice and lead to a dramatic increase in both search and semantic complexity.

Let $A, B \in R$ and let c denote a constant in $\mathrm{dom}(A)$. We allow the following predicate forms: (i) comparisons $u\ \theta\ v$ with $\theta \in \{=, \neq, <, \leq, >, \geq\}$ where u and v are tuple attributes (e.g., $t.A$, $t'.A$, $t.B$) or constants; (ii) pattern predicates $t.A$ LIKE p and $t.A$ NOT LIKE p where p is an SQL pattern; (iii) set-membership predicates $t.A$ IN C and $t.A$ NOT IN C where $C \subseteq \mathrm{dom}(A)$; and (iv) null checks $t.A$ IS NULL and $t.A$ IS NOT NULL.

A DC φ is *violated* if there exist tuples for the quantified variables such that all predicates hold simultaneously.

DCs can be translated directly to SQL. For example, the SQL query in Example 1 is actually equivalent to the following DC:

$$\varphi : \forall t, t' \in \mathit{applications} : \neg\begin{pmatrix} t.\mathit{year} = t'.\mathit{year} \ \wedge\ t.\mathit{edition} = t'.\mathit{edition} \ \wedge \\ t.\mathit{institution_id} = t'.\mathit{institution_id} \ \wedge\ t.\mathit{campus_id} = t'.\mathit{campus_id} \ \wedge \\ t.\mathit{course_code} = t'.\mathit{course_code} \ \wedge\ t.\mathit{shift} = t'.\mathit{shift} \ \wedge \\ t.\mathit{application_mode} = t'.\mathit{application_mode} \ \wedge \\ t.\mathit{score} > t'.\mathit{score} \ \wedge\ t.\mathit{ranking} > t'.\mathit{ranking} \end{pmatrix}.$$

Why DCs. DCs provide a compact formalism for a broad class of integrity constraints, including checks, keys, functional dependencies, order dependencies, conditional variants, and other cross-row conditions. They are also directly checkable on a relation through their violation semantics, making them a natural target for compliance-oriented rule extraction. Compared to SQL, which allows many syntactically different but equivalent formulations of the same rule, DCs offer a more uniform and canonical representation, yielding a more stable target space for extraction and evaluation. Finally, with the standard restriction to at most two tuple variables, the space of candidate constraints remains manageable in practice. Future work can focus on extending the proposed framework to even richer formalisms and/or SQL encodings.

3 System Overview

This section presents `Norma`, our proposed framework for the Text-To-Constraints problem.

Intuitively, one can address this problem by using an LLM with a general prompt that includes the entire document, the dataset schema, and instructions for extracting constraints. However, document-level prompting faces structural challenges on some inputs, as relevant statements may be scattered across the text, much of the content may be unrelated to data quality, and the model must jointly identify relevant parts of the text, align them with the schema, and construct valid constraints. The outputs of this document-level mode also offer less direct traceability, since linking a generated constraint back to the supporting text is itself a separate step.

ALTI: [R1-O4] Tornar o argumento de (falta de) rastreabilidade mais preciso e menos absoluto. O revisor não comprou "LLM não é rastreável". Reformular: não é que o LLM seja incapaz, e sim que o modo document-level NÃO produz por construção o vínculo sentença->constraint; ele exige um passo extra de atribuição. Citar/discutir a alternativa do revisor (pipeline iterativo de sumarização antes do mapeamento) e dizer por que o modo sentence-level já entrega essa rastreabilidade de graça.

We therefore consider an alternative mode and decompose the problem into a pipeline that operates primarily at the sentence level, processing each sentence individually. This design provides fine-grained traceability, since it links each constraint to the specific sentence that justifies it, and allows each component of the pipeline to be tuned independently.

We design `Norma` to support these two modes, and our experiments in Section 6 show that sentence-level processing is the more robust choice overall, particularly when traceability is important.

Norma takes as input a document in natural language and a relational dataset and outputs sentence–constraint pairs in sentence-level mode, or a flat set of constraints in document-level mode. Figure 1 outlines the pipeline.

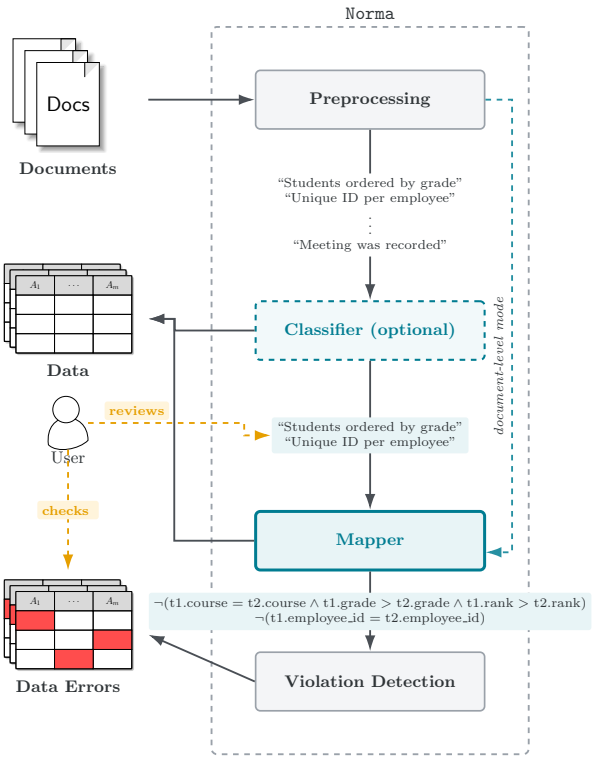


Figure 1: Overview of the Norma pipeline. Solid arrows show sentence-level mode. The dashed blue arrow shows document-level mode. Blue components represent the novel contributions presented in this paper.

Preprocessing. We start from documents (e.g., PDF files) and produce a normalized text representation. This step can be noisy, as headers and footers often leak into the body, line breaks split tokens, footnotes disrupt reading order, and lists get flattened into fragments that hinder sentence boundary detection. Since our goal is Text-to-Constraints rather than document conversion, we treat preprocessing as input preparation and do not focus on it in this work. We run an automated pipeline covering text extraction, dehyphenation, whitespace normalization, and sentence segmentation, followed by minimal manual cleanup.

Constraint classifier. Given the sentences from preprocessing, a lightweight classifier can optionally identify those that likely express checkable constraints, filtering out content that surely has nothing to do with data quality constraints. Since constraints missed at this stage cannot be recovered downstream, we prioritize recall at this stage. False positives can still be ruled out by the mapper, which applies a more expressive classification strategy internally.

ALTI: [R3-O5] Referência adiantada confusa: o "mapper" ainda não foi definido aqui. Aplicar a correção sugerida pelo próprio revisor: trocar "...ruled out by the mapper, which applies a more expressive classification strategy internally" por "...ruled out by the mapper (which we define next)", evitando descrever a "estratégia mais expressiva" antes de o leitor saber o que é o mapper.

When no classifier is used, all sentences are passed directly to the mapper. A user can optionally be involved at this stage, reviewing and correcting the set of classified sentences before proceeding to mapping. We describe the classifier details in Section 4.

Constraint mapper. The mapper is the central and most complex component of Norma. It takes each sentence, optionally pre-filtered by the classifier, and produces a denial constraint grounded in the dataset schema. Some mapper implementations have an internal classification step that uses schema-based information to decide whether a sentence yields a valid constraint before attempting to construct one. The mapper must interpret natural language, align it with schema elements, and synthesize a constraint with correct violation semantics. Outputs that fail syntactic validation are rejected, and sentences that yield no valid constraint are excluded from the final output. In document-level mode, the mapper receives the entire document in a single call and extracts all constraints at once, using similar prompting strategies and schema-grounded instructions as in sentence-level mode. We describe the different mapper implementations in detail in Section 5.

Violation detection. Each constraint is validated by compiling it into a SQL violation query and executing it on the target relation. In sentence-level mode, each constraint is also linked back to the sentence that produced it. Norma returns the violating tuples to the user for further inspection. DC violation detection is a well-studied problem, with specialized systems available for large-scale consistency checks [29, 37, 47].

ALTI: [R1-O6] Justificar a escolha de compilar para query SQL de violação em vez de usar engines especializadas de validação de DC. O parágrafo já cita facet/rapidash/youri; explicitar que SQL/DuckDB é só a implementação do protótipo e que as engines de DC são uma alternativa drop-in para escala, respondendo diretamente à pergunta do revisor.

4 Constraint Classification

Given the segment set $\mathcal{S}$ extracted from a regulatory document, the constraint classifier aims at identifying the subset $\mathcal{S}_c \subseteq \mathcal{S}$ of segments that state data quality constraints. A segment is a positive example if it expresses a condition that the data is expected to satisfy and that can be validated against a dataset. Many segments only define terms, describe procedures, or provide context. Constraint-related segments appear less often, and many segments that sound like rules are still too vague to test with data.

For ease of exposition, we treat each segment as a single sentence from now on, unless stated otherwise.

Example 2 (Restaurant inspections). *The NYC Department of Health publishes restaurant inspection results, including scores*

and letter grades,[4] *alongside a document describing the grading and inspection regulations.*[5] *Table 1 shows sample sentences from these regulations. The first two sentences state data quality rules that can be validated when the dataset contains the needed information. The first connects score ranges to letter grades and can be checked by comparing the score and grade fields. The second constraint defines inspection frequency per establishment and can be checked when establishment identifiers and inspection dates are available. The last two are not data quality rules: one is too underspecified to test because it does not define which inspections yield grades, and the other describes user permissions rather than a condition on the data.* □

Sentence	*Constraint?*
A score of 0–13 results in a grade of A.	*Yes*
Every food service establishment is scheduled for at least one inspection per year.	*Yes*
Only certain inspections result in a grade.	*No*
Consumers may check an establishment's inspection history.	*No*

Table 1: Constraint vs. non-constraint sample sentences from NYC restaurant inspection regulations.

Task Definition	For each sentence, decide whether it states a DATA QUALITY CONSTRAINT that can be validated from the dataset when the required fields are available.
Rule Types	Required/non-null field; Allowed or forbidden values; Numeric bounds; Conditional requirements; Cross-row consistency; Determinacy; Temporal constraints; …
Exclusions	Headings, titles, citations; Definitions or descriptions; Procedures or human decisions; Statements not testable from the dataset; …
Notes	Modal verbs ("may", "should") do not disqualify a sentence; what matters is whether it describes a data condition or verifiable property; …
Examples *(optional)*	YES: "Field X is required." "Two rows with the same Key must not disagree on Field H." … NO: "Field X represents the total amount." …
Context *(optional)*	Table Schema: [Column Names / Types] Table Sample Rows: [Example Tuples]
Input	Sentences: [Numbered List of Sentences] Respond ONLY: "<number>. YES" or "<number>. NO"

Figure 2: Constraint classification prompt structure (simplified). Colors indicate: ■ positive label clues, ■ negative label clues, ■ data context, and ■ instructions.

Norma implements the classifier via *LLM-based prompting*, which can incorporate schema and data samples at inference time and generalizes across domains without requiring labeled training data.

Constraint Classification by Example

We ask a large language model to label each sentence as either stating a data quality rule or not. Figure 2 shows how the prompt is generally structured. Our design is inspired by few-shot in-context learning [11], where a model is guided by a small number of task examples provided in the input context. In our case, these examples are generic and independent of the target domain, serving to guide the model's structured reasoning over candidate constraints. We require the output to be structured as lists and use regular expressions to parse the responses. If parsing fails after several attempts, we mark the sentence as non-constrained. Section 6 compares closed- and open-weight models using these prompt versions.

Prompt structure. Following standard practice in prompt-based text classification [59], we include a task definition, category descriptors, and examples. These elements are essential because constraint classification presents two main challenges. The first challenge is that the positive class is rare and spans many syntactic forms. To reduce false negatives, we enumerate rule types that the model should treat as constraints. Otherwise, it may miss cases such as functional dependencies that might use specific language. The second challenge is that non-constraint sentences can be easily mistaken for constraints. Definitions, procedures, and contextual statements often use similar wording, mention data-related concepts, or contain keywords that suggest a requirement. To reduce this confusion, we explicitly add notes on exclusions and edge cases.

Examples. Our approach follows a few-shot prompting strategy [11], using generic templates that remain fixed across datasets, rather than domain-specific examples. As a result, the prompt remains portable without requiring labeled data.

Table Context. We evaluated multiple prompt variants. Some ask the classifier to decide from the sentence alone whether it expresses a checkable constraint. Others also provide table context, including table and column information and sample tuples. Without access to the schema, the classifier may produce false positives for sentences that cannot be checked against the available fields. Schema context helps reject such cases and, therefore, mainly improves precision. However, it also increases prompt length and can make predictions sensitive to the particular sample tuples provided.

Batch processing. To reduce API overhead, Norma classifies sentences in batches, concatenating multiple sentences into a single prompt using numeric indices and parsing one label per index from the response. We use moderate batch sizes (up to 10 sentences per prompt), as larger batches increase prompt length and the risk of truncated outputs and parsing errors. Malformed responses trigger a retry; if the model still fails, we fall back to processing the remaining sentences individually.

5 Constraint Mapping

Let $\mathcal{S}$ denote the sentences passed to the mapper, either all preprocessed sentences or the subset retained by the classifier. For each $s_i \in \mathcal{S}$, the mapper returns a denial constraint (DC) $\varphi_i = \neg(p_1 \wedge \cdots \wedge p_\ell)$ over the dataset schema, or no output if s_i does not express a constraint or cannot be linked to the schema.

Recall from Section 2 that DC predicates can take several forms, including comparisons between an attribute and a constant (e.g., `t.A = c`), between two attributes of the same tuple (e.g., `t.A > t.B`), or between attributes across tuples (e.g., `t1.A = t2.A`), as well

[4] Dataset available at https://data.cityofnewyork.us/Health/DOHMH-New-York-City-Restaurant-Inspection-Results/43nn-pn8j. See Section 6 for details.
[5] Available at https://www.nyc.gov/site/doh/services/restaurant-grades.page.

as null checks and membership tests over permitted values. The mapping task is therefore to interpret a sentence, identify the relevant schema elements, select the predicates on those elements that capture the constraint intuition, and produce an executable *violation condition*. In this sense, it is related to Text-to-SQL [61], as both tasks aim to connect a natural language segment to schema-aware logical expressions. One key difference is that Text-to-SQL typically retrieves satisfying records, whereas our target is a conjunction of predicates that identify a constraint violation.

Example 3 (Mapping to DC predicates). *Consider the sentence "A score of 0–13 results in a grade of A" from the NYC restaurant inspection regulations. This sentence states that scores in the range 0–13 must be associated with grade A. The corresponding violation condition is:*

$$\texttt{t.score} \geq 0 \land \texttt{t.score} \leq 13 \land \texttt{t.grade} \neq \text{`A'}$$

Any tuple satisfying all three predicates violates the rule. □

Mapping regulatory text to DCs is challenging because the same rule may be expressed in many different ways, often with implicit references to schema elements, literals, or tuple relationships.

ALTI: [R2-O2, R2-O3, R1-O5, R3-O2] PONTO CRÍTICO (queixa dos 3 revisores). Adicionar uma subseção dedicada de "novidade metodológica" articulando a contribuição ALÉM dos prompts. Tese de defesa: o cerne não são os prompts, mas (a) o espaço de predicados controlado derivado do schema que impede colunas alucinadas/operadores não suportados, (b) a forma de violação (não-satisfação) como alvo, e (c) os checks pós-construção/orphan-predicate sobre os dados. Fazer uma tabela explícita delimitando, componente a componente, o que é ADAPTADO de Text-to-SQL (schema linking, decomposição em estágios, como RESDSQL e DIN-SQL) vs. o que é NOVO (espaço de predicados como vocabulário controlado, forma de violação, grounding por orphan-predicate). Sem isso, paper é lido como "engenharia de prompt".

We therefore rely on LLM-based mappers, motivated by their strong performance on related semantic parsing tasks such as Text-to-SQL [56]. However, unrestricted LLM generation is unreliable in our setting, it is known that LLMs can hallucinate column names and use unsupported operators [19, 22, 56]. We also observed that LLMs tend to output predicates describing valid states rather than violations, so we introduce hints toward violations in the prompt.

To make mapping tractable and reliable, our design follows two principles. First, we derive a *predicate space* from the dataset schema and use it as a controlled vocabulary. The mapper must either select predicates from this space or instantiate typed placeholders within predefined predicate templates. This reduces the search space and prevents ill-formed outputs. Second, the mapper is biased toward *precision*. Incorrect mapping can generate false violations and corrupt downstream checks, whereas an unmapped sentence can still be marked for manual inspection.

We implement three mapper variations that share the same predicate space but differ in how they use it. The *single-prompt mapper* (Section 5.2) and *multi-step mapper* (Section 5.3) operate at the sentence level: the former performs the full mapping in a single LLM call, while the latter decomposes the task into focused LLM calls interleaved with deterministic checks. The *document-level mapper* (Section 5.4) operates at the document level in a single LLM call, without native sentence-to-constraint traceability.

5.1 Predicate Space

Traditional DC discovery operates over a *predicate space*, that is, the set of predicates allowed to appear in candidate constraints [15, 46]. Because discovery methods enumerate and validate combinations of predicates, this space is kept small, which limits expressiveness [15].

Our setting requires a richer predicate space. Regulatory text frequently refers to explicit literals, enumerations, ranges, nullability conditions, and string formats that are not covered by traditional DC discovery spaces.

ALTI: [R3-O7] Corrigir a alegação: o revisor apontou (com razão) que DCs tradicionais JÁ cobrem null checks e comparação com literais/constantes. Restringir a frase ao que realmente falta no espaço clássico: ranges, enumerações (IN) e padrões de string (LIKE). Reescrever para "...ranges, enumerations, and string formats not covered by traditional DC discovery spaces", removendo "explicit literals" e "nullability conditions" da lista de ausências.

We therefore extend the predicate space with typed predicate templates that can be instantiated with literals extracted from text. Importantly, `Norma` does not enumerate candidates DC over this larger space. Instead, it uses the space as a controlled vocabulary for mapping, which avoids candidate explosion while preserving syntactic validity.

Predicate templates. We derive a finite set of predicates and predicate templates from the schema, while the data instance is used to support specific instantiations and validation steps. We consider the following categories:

(1) **Cross-tuple predicates** compare attributes across two tuples (e.g., `t1.A = t2.A`, `t1.A > t2.B`). We allow $\{=, \neq\}$ for type-compatible attributes and additionally $\{<, \leq, >, \geq\}$ for numeric attributes.
(2) **Same-tuple predicates** compare two attributes within one tuple (e.g., `t1.A` $\leq$ `t1.B`), capturing intra-tuple rules.
(3) **Null predicates** (`t1.A IS NULL`, `t1.A IS NOT NULL`) capture requiredness and conditional missingness constraints.
(4) **Attribute–constant predicates** compare an attribute to a constant c (e.g., `t1.A = c`, `t1.A` $\geq$ `c`).
(5) **Pattern predicates** encode string-format and substring conditions (e.g., `t1.A LIKE p`).
(6) **Set predicates** encode membership in small enumerations (e.g., `t1.A IN C`).
(7) **Arithmetic predicates** compare an arithmetic expression over numeric attributes of the same tuple with an attribute or a constant (e.g., `t1.A` $\geq$ `c * t1.B`), when the schema enables arithmetic for those attributes.

For categories 4–7, the mapper instantiates the placeholders c, p, or C using literals extracted from the sentence.

ALTI: [R2-O4] Adicionar (i) análise de COBERTURA destas 6 categorias contra o conjunto de gold constraints de cada dataset (quantas regras-alvo são expressáveis), e (ii) discussão explícita dos limites de expressividade, em particular a exclusão de agregações e aritmética temporal, que forçou derivar manualmente a coluna "days-until-next-inspection" no Restaurant. Assumir essa limitação como escopo, não escondê-la.

Predicate restrictions. To avoid semantically uninformative or implausible predicates, we adopt a filtering principle from DC discovery [15]. Order predicates are generated only for numeric attributes. Cross-tuple and same-tuple predicates are generated only for type-compatible pairs and are further pruned using observed value overlap, helping eliminate comparisons between unrelated columns. Nonetheless, our extensions with predicate templates, constants, patterns, and sets expand the predicate space from the dozens common in DC discovery to several hundred in some datasets. This space is impractical for traditional data profiling, where the enumeration cost grows exponentially with the number of predicates. In our approach, the larger predicate space remains practical because we use it only as a constrained target language for mapping, not as a search space for enumerating combinations.

Schema-driven, on-demand predicate space. Materializing the full space does not scale to wide schemas: with arithmetic and code-set templates, the space of a 135-column table reaches hundreds of thousands of templates, most of which are irrelevant to any given sentence. `Norma` therefore stores only a *schema description* and generates predicates on demand. For each column, the description records its type, the comparison and arithmetic operators it admits, its domain values when it has at most a few distinct values, and a short natural-language description. Descriptions are written once, offline, by an LLM from a deterministic profile of the data (type, null rate, distinct values, ranges, frequent values, format patterns, and observed code–label pairs); when an official data dictionary is available, its definitions are merged into the description of the corresponding columns (we do this for HMDAZ, using the FFIEC field definitions). At mapping time, once the relevant columns of a sentence are selected, the predicates of the categories above are generated only for those columns, under the same restrictions. For a fixed instance and fixed options, the generated set equals the full space restricted to the selected columns, so the controlled vocabulary is unchanged; only its materialization is deferred.

5.2 Single-Prompt Mapper

The single-prompt mapper performs the full mapping in a single LLM call. We design the prompt around three elements: the schema and predicate space as a controlled vocabulary, a small set of examples covering the main constraint patterns, and predicate specifications to guide the model in understanding and mapping violation conditions. Given an input sentence, the model returns either a list of predicates drawn from the predicate space or `null`. Figure 3 summarizes the prompt structure. In the current version, the prompt is restricted to the columns retrieved for the sentences of each batch (three sentences per call): columns are ranked by the embedding similarity between the sentence and the column name and description (semantic retrieval), and the prompt carries the description and domain values of each retrieved column (column context) together with the predicates generated for them. The model must also state, for each sentence, the condition under which a tuple violates the rule (`violated_when`) before writing the DC, which makes the violation form explicit. Rules that reference unknown columns or compare a column with a constant of the wrong type are returned to the model once, with the error message.

Definition	A denial constraint (DC) is a conjunction of predicates that *cannot* all hold simultaneously in a dataset. DC predicates must encode *violations* of data quality rules, not valid states.
Violation Form	"must be X" ⇒ `!= X`; "must not be X" ⇒ `= X`;
Cross-Tuple	Signals such as "same …", "repeated", or "consecutive" introduce `t1`, `t2`, and a join predicate on the shared key.
Context	Table schema + top-k frequent values per column. List of possible predicates (predicate space).
Examples	"Value must be at least 100" ⇒ `["t1.value < 100"]` "Same ID must share status" ⇒ `["t1.id = t2.id", "t1.status != t2.status"]` "The system performs well" ⇒ `null`
Output	Array of length n, one entry per input sentence; `null` for non-constraints. Predicates must come from the predicate space.

Figure 3: Prompt structure for the single-prompt mapper. Examples are representative, as the full prompt covers threshold, set-membership, field-comparison, null, cross-tuple, and non-constraint cases. The colors represent: ■ Data context; ■ constraint example; ■ non-constraint; ■ instructions.

Violation form. Without explicit guidance, LLMs generally tend to output predicates describing *satisfying* tuples rather than *violating* ones. Since regulatory sentences typically describe what must hold, the mapper must find predicates expressing a violation condition—as in Example 3, where the correct output is `grade ≠ 'A'`, not `grade = 'A'`. We make this criterion explicit in the prompt through a small set of examples.

Cross-tuple constraints. Some rules address relationships between records rather than conditions within a single record. In these cases, the mapper must introduce two tuple aliases and a join predicate on the shared identifier.Including cues such as "for the same" in the prompt already suffices for the model to generalize reliably to the variety of phrasings in which such predicates appear.

ALTI: [R1-O8] Esclarecer o escopo das constraints cross-tuple: o revisor perguntou como condições que valem para todas/maioria das tuplas (estilo differential dependencies) são encontradas e validadas. Deixar explícito que o alvo são DCs com no máximo 2 variáveis de tupla, com semântica universal de violação (sem agregação/contagem sobre maioria), e que differential dependencies ficam fora do escopo. Se forem suportadas, explicar o mecanismo de validação.

Literal grounding. Many constraints require grounding numeric thresholds, strings, or small enumerations. A recurring failure is to

produce literals in the wrong format or casing. To mitigate this, we include in the prompt the top-k frequent values per column as a domain context, which anchors literals to the dataset representation. In practice, we found that fewer than ten samples per column usually suffice for the model to capture the relevant patterns.

The mapper returns at most one DC per sentence. If the sentence is not mappable, it returns `null`. We parse the output and validate every predicate against the schema: it must reference existing columns, use operators admitted by their types, compare them with constants of the right type, and have no uninstantiated placeholders. Membership in the generated predicate space is not enforced for this mapper; we report it instead (out-of-space predicates). Malformed outputs are retried a fixed number of times, and invalid rules are returned once to the model with the error. If all attempts fail, the sentence produces no constraint. To prevent incorrect mappings from affecting downstream checks, we intentionally keep this stage conservative.

Batch processing. As in the classifier (Section 4), we support batch processing by concatenating multiple indexed sentences into one prompt and returning one output per index.

5.3 Multi-Step Mapper

The single-prompt mapper needs to parse several subtasks in one call: schema linking, predicate selection, literal linking, and violation form. In this setting, errors in early steps might sometimes propagate. For example, a wrong column choice may break predicate selection, a hallucinated literal yields a syntactically valid but semantically meaningless constraint, and a non-constraint sentence triggers an unnecessary mapping attempt. Recent work in Text-to-SQL shows that decomposing semantic parsing into multiple stages, especially separating schema linking from later query building decisions, often improves accuracy [19, 22, 53, 56].

ALTI: [R2-O5] Justificar POR QUE quatro estágios e adicionar ablação mais fina do que Single-vs-Multi: remover/reordenar estágios (p.ex. sem rule-filter, sem schema-linking separado, ordem trocada) para mostrar que o ganho vem da DECOMPOSIÇÃO e não apenas de mais chamadas ao LLM. Caso contrário o ganho do Multi-Step é atribuível a "mais compute".

We propose a multi-step mapper that breaks down mapping into several LLM calls, with deterministic checks in between. The process uses four LLM calls, where each call receives the output from the previous step. Sentences that cannot be mapped are removed before constraint construction. After construction, data-driven checks eliminate constraints that have incorrect literals.

Figure 4 illustrates the control flow of our multi-step mapper and Figure 5 provides a summary of its prompts in each step.

Schema linking. This step identifies the columns relevant to the sentence, including implicit references and derived concepts. We prompt the model to return the set of columns relevant for the sentence, since operating over a smaller, relevant subset reduces the risk of incorrect predicate selection in the later step. If no column with defined predicates can be aligned to any concept in the sentence, we reject the sentence for mapping.

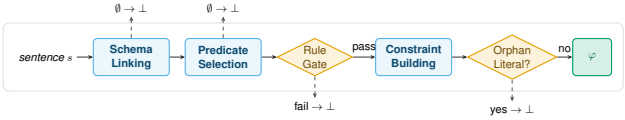


Figure 4: Multi-step mapper control flow.

Step	Context	Task
Schema	Full schema, column profiles, predicate space	Identify all columns relevant to s: direct mentions, synonyms, and derived concepts. Abort if no concept maps to any column with defined predicates.
Predicates	↪ Columns from Schema step Predicate space restricted to those columns Column profiles	Select predicates and predicate templates from the restricted predicate space and identify the constraint pattern for s (value mapping, temporal bound, functional dependency, required field, or conditional rule). Abort if no predicate or predicate template fits.
Rule filter	↪ Columns and predicates from Predicates step	Decide whether s expresses a specific, data quality rule verifiable from column values alone. Reject: process descriptions, permissions, vague quantifiers, physical conditions. Accept: value mappings, temporal bounds, functional dependencies, required fields, conditional rules.
Construction	↪ Predicates and predicate templates from Predicates step Column profiles	Substitute predicate placeholders with literals from s and return the constraint in violation form: a conjunction of predicates that must not hold simultaneously.

Figure 5: Prompt structure of the multi-step mapper. Context narrows progressively. The rule filter runs after predicate selection but before construction, avoiding unnecessary LLM calls on non-rules. ■ input context (variable per step); ■ accept signal; ■ abort / reject signal; ■ fixed instruction.

Predicate selection. The mapper uses only columns retained by schema linking. Given the column names, profiles, and restricted predicate spaces, it selects predicates and predicate templates (requiring literals) that together form the DC pattern. We separate column selection from predicate selection to address distinct subproblems: schema linking determines which attributes are relevant to a sentence, while predicate selection determines how those attributes are combined into a denial-constraint structure. This decomposition reduces the DC structural search space and helps avoid predicates that, while individually plausible, do not compose into a valid constraint. This design aligns with prior results in text-to-SQL, where having a dedicated structural parsing step has been shown to improve accuracy [31].

Rule filter. This step determines whether the sentence can be mapped to a DC under the schema-grounded interpretation obtained so far. Unlike the classifier in Section 4, the rule filter has access to the candidate columns and predicates selected for the sentence. This improves the semantic reliability of the decision, since many sentences that appear rule-like in isolation may fail to

support a coherent executable constraint once their concepts are grounded in the schema and tied to candidate predicates.

Constraint building. This step instantiates the selected predicate templates with the literals needed to form the final DC. At this point, the candidate columns and predicate types have already been fixed, so the task is no longer to explore the full predicate space, but to realize a small set of grounded predicate instances. The model resolves value-level details such as numeric thresholds, categorical constants, intervals, string patterns, and short enumerations, and aligns them with the dataset representation using column profiles and sample values. This grounding is important because the text and the data often use different representations. A sentence may describe a concept in natural language, while the table records it as an abbreviation, code, or differently formatted value. The output is a set of predicates in violation form, which are then assembled into the final DC. Framing this stage as grounded instantiation rather than open-ended generation substantially narrows the construction problem and makes unsupported predicates and malformed expressions less likely [34, 50, 54].

Post-construction checks. After constructing a candidate DC, both sentence-level mappers apply a set of validation checks to ensure that the generated constraints are both executable and semantically plausible. First, we verify the structural validity of the DC: each predicate must belong to the predicate space, reference valid schema columns, use type-compatible operators, and contain no uninstantiated placeholders. Next, we apply a data-based grounding check for unsupported literals. For each predicate with literals involving equality, string inequality, `LIKE`, or set membership, we issue an existence query against the dataset. If no tuple can satisfy the predicate, we refer to it as an *orphan predicate*, i.e.,, a predicate that is syntactically valid but unsupported by the data instance. Any constraint containing a orphan predicate is disregard for validation, although it may be retained for documentation purposes. Numeric bounds and `null` predicates are excluded from this check because their validity is structural rather than value-based. If parsing fails, the mapper retries a small fixed number of times; if all attempts fail, no constraint is returned.

5.4 Document-level mapping

The mappers described above operate at sentence granularity, which is the primary focus of this work. As noted in Section 2, the Text-to-Constraints problem admits other granularities: a segment may span multiple sentences, a paragraph, or an entire document, and a single segment may yield multiple constraints. We also integrate a document-level mapper into `Norma` to study this alternative granularity. While sentence-level processing is the primary focus of this work, document-level mapping serves as a useful complementary mode, and a broader study of granularity choices and sentence-grouping strategies is left for future work.

This setting introduces additional challenges, as regulatory documents contain substantial amounts of text that are not directly related to data quality constraints, including headings, definitions, explanatory passages, and procedural descriptions.

The document-level mapper submits the entire document to the LLM in a single call and returns a flat set of DCs, where the DC count is determined by the document content rather than the number of input sentences. It reuses the predicate space, violation-form instructions, and literal-grounding conventions developed for the sentence-level mappers, so the controlled vocabulary and instructions carry over without modification.

The prompt used in this mode differs from the single-prompt mapper (Figure 3) primarily in the output format, replacing sentence-indexed predictions with an unordered list of constraints. As before, every returned predicate is validated against the predicate space and discarded if it references unknown columns, uses unsupported operators, or contains uninstantiated placeholders.

6 Experimental Evaluation

We implemented a prototype of `Norma` and evaluated it on several datasets prepared for the *Text-to-Constraints* setting. Our evaluation is organized around the following questions.

- **RQ1: Constraint-sentence classification.** How accurately can `Norma` identify which sentences in a normative document express data constraints?
- **RQ2: Mapping quality.** Given constraint-relevant text, how accurately can `Norma` map it to executable denial constraints and how well do these constraints capture the corresponding data violations?
- **RQ3: Impact of implementation choices.** How do different classifier and mapper implementations, LLM backends, and prompt variants affect extraction quality and robustness?
- **RQ4: Comparison with baselines.** How does `Norma` compare with data profiling and Text-to-SQL baselines on constraint extraction and violation detection?

6.1 Datasets

We evaluate `Norma` on six datasets: four based on real-world regulatory documents paired with corresponding tabular datasets, and two derived from benchmark datasets commonly used in data quality and profiling research. A summary of these datasets is provided in Table 2, including the number of rows and columns, the number of sentences in the source text, the number of annotated constraints, and the number of tuples violating at least one constraint.

Dataset	#Rows	#Cols	Sent.	Const.	Viol.
Hospital	1k	17	109	15	834
Restaurant	128k	21	33	7	52,311
SISU	102k	56	42	17	487
Tax	100k	15	126	6	468
HMDAZ†	1M	135	17	9	209
HMDA Checks†	1M	81	158	92	986,491

Table 2: Dataset statistics. †First 1M rows of the public file (15.1M and 4.8M rows); *Const.* counts sentences mapped to a non-null gold DC.

SISU. As discussed in Section 1, the SISU dataset contains admission records from Brazil's centralized university selection system. We manually annotated three sections of *Portaria Normativa No. 21/2012* covering student registration, candidate classification and selection, and waitlist procedures. The resulting constraints span nullability

checks, functional dependencies, ordering constraints, and conditional selection rules. Many have violations in the released data, suggesting inconsistencies between the published records and the regulatory requirements expressed in the text.

ALTI: [R1-O10] Detalhar a construção do ground truth: (i) como as gold constraints foram derivadas/anotadas e por quem, (ii) similaridade e redundância entre sentenças (há sentenças quase duplicadas?), e (iii) classificar os tipos de constraint do SISU em formalismos conhecidos: ordering = order dependencies (ODs), conditional selection = conditional FDs (CFDs)? O revisor pediu esse mapeamento explícito.

Restaurant. As introduced in Example 2, this dataset contains restaurant inspection records published by the Department of Health and Mental Hygiene of New York. We annotated the NYC food service operator inspection guide, focusing on the grading and closure sections, which present constraints on score-to-grade mappings, inspection frequency, grade posting timing, and closure conditions. Most have natural violations and can be grounded directly to existing fields. For the one constraints that reasons over inspection sequences, we added one derived column that tracks the days until the next inspection. We leave to future work how `Norma` can guide or automate such dataset transformations more systematically.

Tax and Hospital. Both datasets have been widely used in data profiling and denial-constraint research since the seminal work of Chu et al. [15]. For Tax, we extracted the constraint descriptions from the original paper [15]. For Hospital, we used the data and constraints from a well-known cleaning system [52]. Because their constraints are available in formal form, we first generated natural-language descriptions with an LLM. For both datasets, we then prompted an LLM to generate additional non-constraint sentences around each rule, mimicking a regulatory document that interleaves binding rules with other prose. The prompt receives the schema and is instructed to use schema terminology, making sentence classification more challenging. To avoid circularity between data generation and evaluation, we used Claude 4.5 Sonnet, a model not evaluated in any `Norma` configuration.

ALTI: [R3-O3, R2-O6.2] Assumir explicitamente a limitação destes dois datasets: o "texto regulatório" vem dos papers que já enunciam as regras, e as sentenças são geradas por LLM, compartilhando priors com os backbones avaliados. Isso infla os scores (Hospital F1=1.0) e superestima a dificuldade real. Acrescentar uma frase deixando claro que SISU e Restaurant (pares texto-tabela realistas, F1 perto de 0.5) são o sinal mais confiável, e que Tax/Hospital servem como cenário controlado.

HMDAZ and HMDA Checks. Both datasets use public loan-level records released under the Home Mortgage Disclosure Act (HMDA) and are, by far, the largest in our evaluation; we use the first 1M rows of each file. *HMDAZ* pairs the 2018 loan application register (135 columns, including the code and label versions of each categorical field) with Sections 1026.32, 1026.35, and 1026.43 of Regulation Z, which define high-cost (HOEPA) and higher-priced mortgages, the limitations that apply to them (e.g., balloon payments, negative amortization, prepayment penalties), and the ability-to-repay requirements. Sentences are verbatim from the regulation, and gold DCs keep the coverage conditions of each trigger (e.g., a loan is high-cost only if it is a closed-end, principal-residence loan). Scope-only text, or text whose outcome is not observable in the data, is annotated with a `null` constraint. *HMDA Checks* pairs the 2014 register (81 columns) with the 2014 HMDA edit specification published by the FFIEC: one entry per edit, with the edit test verbatim; edits that cannot be checked on a single record (aggregate, transmittal-sheet, or external-reference edits) are `null`. These datasets stress three aspects that the original four do not: wide schemas whose relevant column is often named after a code rather than the concept in the text, rules that compare columns with each other or through arithmetic (e.g., loan amount versus five times the income), and a highly concentrated violation distribution (Section 6.3).

6.2 Baselines

Since no prior system addresses the Text-to-Constraints task directly, we compare `Norma` against the two closest paradigms: DC discovery from data alone and Text-to-SQL.

ALTI: [R1-O12] Considerar baselines além de DC discovery, já que reivindicamos cobrir ordering e conditional selection: incluir descoberta de ODs (order dependencies) e CFDs como comparativos, ou justificar por que DC discovery aproximada já subsume essas classes no nosso setting. Conectar com a classificação de tipos de constraint pedida em [R1-O10].

Data profiling. We use the DC discovery algorithms from [48], running both exact and approximate denial-constraint discovery. Exact discovery returns only constraints that hold on the data with no violations. As a result, it is not suitable for our violation-based evaluation, which aims to recover rules that may be violated in the released dataset. We therefore focus on approximate DC discovery as the main profiling baseline. Approximate DCs allow a small fraction of violating tuple pairs, controlled by a threshold ε, and can therefore recover rules even when the data is not fully clean.

Because discovery can return many DCs, we rank them using the standard criterion from prior work [15], which combines coverage and succinctness, with succinctness measured as the inverse of the number of symbols in the DC. We run a grid over multiple top-k values and ε thresholds, and report the best results obtained across these settings, following standard practice in prior work.

These baselines do not scale to our largest schemas. In particular, they cannot process SISU or Restaurant with all columns present. For those two datasets, we therefore restrict the input to the subset of columns that appear in the gold constraints, giving the profiling baselines the best possible setting under their scalability limits.

Text-to-SQL. We use XiYan-SQL [36], a state-of-the-art generative text-to-SQL framework. We adapt it to our setting by prompting it, for each sentence, to return a SQL query that identifies the tuples violating the described constraint, or `NULL` if the sentence does not express a checkable data constraint. Under this formulation, XiYan-SQL acts jointly as a constraint classifier and mapper, making it directly comparable to `Norma` on across all evaluation dimensions.

6.3 Evaluation Metrics

Text-to-Constraints requires solving three related problems: identify text that expresses constraints, translate those into a formal representation, and identify the data violations implied by those constraints. We therefore evaluate systems along three complementary dimensions: *Constraint Classification*, *Constraint Extraction*, and *Violation Detection*. This is similar in spirit to Text-to-SQL evaluation, where execution-based evaluation complements structural matching because exact form alone does not fully capture correctness [65]. Because `Norma` and the baselines in Section 6.2 differ in what they produce, not all dimensions apply to every method.

Constraint Classification. This dimension measures whether a system correctly identifies which sentences in a document express constraints. We report precision, recall, and F_1 over all sentences. This dimension applies to sentence-level variants of `Norma` and to the Text-to-SQL baseline. It does not apply to data profiling baselines, which operate on data rather than text, or to the document-level variant, which does not produce sentence-level predictions.

Constraint Extraction. This dimension measures how accurately systems translate text into denial constraints. We compare predicted and ground-truth constraints by predicate-set match (i.e., exact matching over the set of predicates), and report precision, recall, and F_1. For `Norma` and the data profiling baselines, the predicate set is read directly from the predicted denial constraint. For Text-to-SQL, we extract predicates from the `WHERE` clause of the generated query. Because different SQL formulations may express the same underlying constraint, this metric can underestimate Text-to-SQL performance, which further motivates reporting the other evaluation dimensions. For data profiling, discovered constraints are matched directly against the ground-truth set.

Violation Detection. This dimension measures whether the constraints produced by a system, when executed on the dataset, identify the same tuples as the ground-truth constraints. Because a denial constraint may involve either a single tuple or a pair of tuples, we evaluate violations at the tuple level by counting tuples that participate in at least one violation. We execute all predicted constraints and all ground-truth constraints on the dataset, then compare the resulting violation sets using precision, recall, and F_1. This dimension applies to all `Norma` variants and all baselines.

Complementary mapping metrics. Exact predicate-set matching penalizes logically equivalent rewritings, and the tuple-level aggregate can be dominated by a single rule. We therefore report four complementary measures for the mappers. (i) *Equivalence-aware extraction* ($CE_{\equiv}$) compares DCs after a logical normal form that expands `IN`/`NOT IN`, orients and sorts operands, flattens and distributes `AND`/`OR`, and removes absorbed terms; in addition, an equality literal on a code column and the equality literal on its label column (e.g., `hoepa_status = 1` and `hoepa_status_name = 'High-cost mortgage'`) are unified only when the data confirms that both select exactly the same tuples. Code–label pairs are discovered in the data, not from column names. (ii) *Execution equivalence* is the fraction of gold rules with at least one violation whose predicted DC flags exactly the same tuples. (iii) *Per-rule violation F_1* averages, over gold sentences, the F_1 between the tuples flagged by the gold and the predicted DC, so that every rule weighs the same. (iv) *Violation concentration* is the share of the gold-violating tuples flagged by the single largest gold rule; it is a property of the dataset, not of the method. We also count predicted DCs whose SQL fails to execute, instead of treating them as producing no violation.

6.4 Implementation and Execution Settings

`Norma` is implemented in Python, with DuckDB as the query execution backend. All LLM calls are issued through OpenRouter with strict JSON-schema structured outputs. We evaluate six backbones: GPT-6 Luna, DeepSeek-V4-Flash, Mistral Large 4, Qwen3.8-2.4T-A95B, GPT-6 Sol, and Claude Sonnet 5.5. Before the runs, every backbone passed a health check of the classifier, both single-prompt output schemas, and each Multi-Step schema. Two candidates were replaced: Kimi K2.6, which repeatedly emitted whitespace until the token limit under strict JSON-schema decoding, and MiniMax-M3, which returned empty single-prompt outputs in the experiments (Section 6.6). Each configuration is executed once, with a cap of 32k output tokens per call. Unless stated otherwise, `Norma` uses the schema-driven predicate space of Section 5.1 with on-demand predicate generation, and the single-prompt mapper uses semantic column retrieval and column context (Section 5.2). Constraints are executed on the full datasets (first 1M rows for HMDA) with DuckDB. `Norma` experiments were executed on a Linux workstation with an Intel Core i7-7700HQ CPU and 16 GB of memory; the DC discovery baselines were executed on a MacBook Pro with an Apple M1 chip and 16 GB of memory. The XiYan-SQL baseline, which requires GPU inference, was executed on a server with an NVIDIA A100 SXM GPU (80 GB VRAM), 125 GB RAM, and 16 vCPUs.

6.5 Classifier Performance

Our first experiment evaluates how model and prompt design affect the accuracy of constraint classification. Starting from the prompt in Figure 2 (Section 4), we derive four variants by enabling or disabling *constraint examples* and *table context*: *No Context*, *Table Context*, *Examples*, and *Examples + Table Context*. We evaluate all four variants across six LLMs and all six datasets. Figure 6 summarizes the results. Because sentences missed at this stage cannot be recovered later by the mapper, we require a recall of at least 0.90 on every dataset when selecting a default configuration (red dashed line).

On Hospital, Tax, SISU, and HMDA Checks, almost every configuration reaches recall 1.00, so the differences lie in precision. HMDAZ and Restaurant behave differently: there, recall is the bottleneck. On HMDAZ, most configurations recover only 4 of the 9 constraint sentences (recall 0.44). The missed sentences are regulatory definitions (e.g., when a mortgage is high-cost) that read as scope statements, although each implies a checkable condition on the data. Only three configurations satisfy the recall requirement on all six datasets: *Qwen3.8-2.4T-A95B* with *Examples* (mean F_1 0.69), *GPT-6 Luna* with *Examples* (0.69), and *Qwen3.8-2.4T-A95B* with *No Context* (0.68). As in our previous evaluation, the qualifying configurations are dominated by *Examples*, and we select *Qwen3.8-2.4T-A95B* with *Examples* as the default classifier. Configurations with higher mean F_1 exist (e.g., DeepSeek-V4-Flash with *Examples + Table Context*, 0.82), but they miss more than half of the HMDAZ constraints.

Table context has two opposite effects. It is the only signal that raises precision on HMDA Checks: without it, the models label

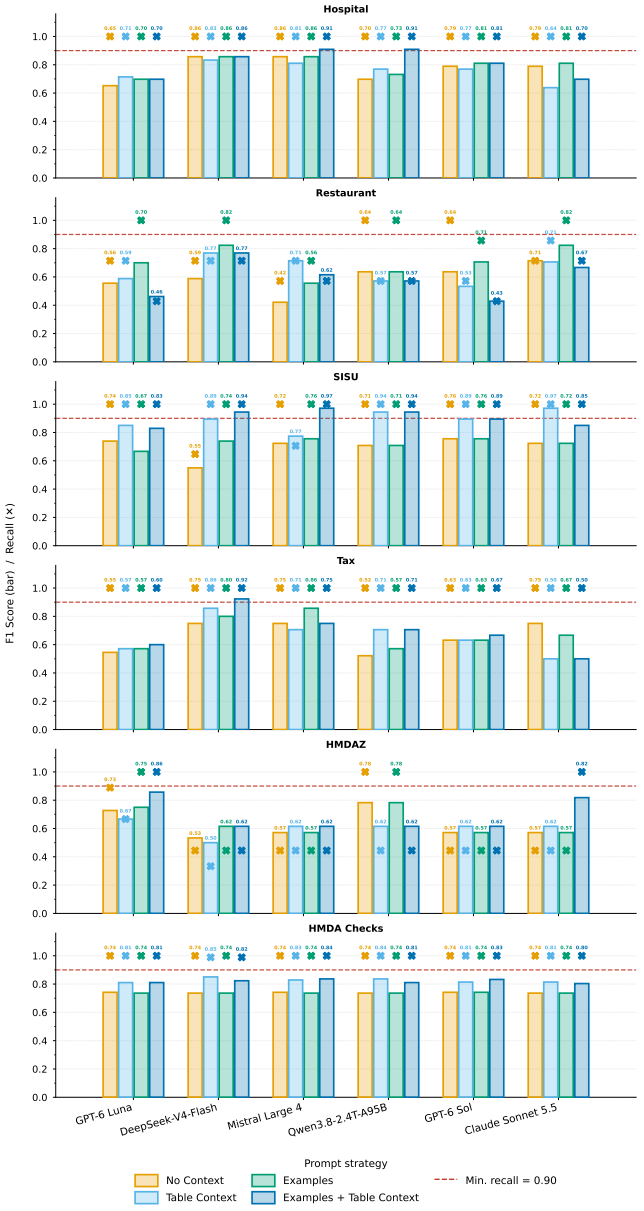


Figure 6: Constraint classification F_1 (bars) and recall (x) across models and prompt variants. The dashed line marks the 0.90 recall threshold.

(almost) every edit as a constraint, and F_1 equals that of the trivial all-positive classifier (0.74). The non-constraint edits are phrased exactly like the checkable ones and differ only in that they refer to aggregates, the transmittal sheet, or external reference data; we conjecture that the record layout shown in the table context is what lets the model tell them apart. With table context, F_1 rises to 0.80–0.85. On wide schemas, however, the same context does not help recall: on HMDAZ (135 columns), 9 of the 12 table-context configurations stay at or below 0.44, and only GPT-6 Luna and Claude Sonnet 5.5 with *Examples + Table Context* recover all nine constraint sentences.

Averaged over all models and datasets, *Examples* gives the best recall (0.93), while the table-context variants give higher precision (0.71 vs. 0.58–0.62) at the cost of recall (0.85–0.87).

Compared with the backbones of our previous evaluation, the current models are, on average, better classifiers on the four original datasets (mean F_1 0.72 vs. 0.69 across all configurations), but the best configuration that satisfies the recall constraint is weaker (0.66 on those four datasets vs. 0.85 for Qwen3-32B with *Examples*). Newer models tend to accept more sentences as constraints, which keeps recall high but lowers precision, especially on Tax and Restaurant.

ALTI: [R3-O4] Analisar POR QUE o classificador já vai bem sem contexto/exemplos (ganhos marginais, sobretudo no GPT). Discutir a hipótese do revisor: possível exposição a esses datasets/texto regulatório no pré-treino. Mitigar/avaliar: comparar datasets sintéticos vs. SISU/Restaurant, e relacionar com a discussão de vazamento de priors em [R2-O6.2].

6.6 Constraint Mapping Performance

Our second experiment evaluates how model and mapper choices affect mapping quality. We evaluate mapping quality using *constraint extraction* F_1 and *aggregate violation* F_1 as metrics. To isolate mapper quality from classification errors, both metrics are computed only on the ground-truth constraint sentences. We compare the *Single-Prompt* and *Multi-Step* mappers across six LLMs on all six datasets.

Figure 7 summarizes the results. On the four original datasets, *Multi-Step* again improves both constraint extraction and aggregate violation recovery over *Single-Prompt* (mean over models: 0.80 vs. 0.75 and 0.90 vs. 0.55). The largest gains are on Restaurant and SISU, whose rules combine several conditions or relate two tuples. Hospital and Tax are easy for most configurations. With the current backbones, the best Multi-Step configurations exceed the best configuration of our previous evaluation (GPT-5, 0.78 extraction and 0.95 violation): Claude Sonnet 5.5 reaches 0.84 and 0.98, and GPT-6 Sol and Mistral Large 4 0.81 and 1.00. Decomposition also keeps the mapper usable when a backbone is unstable under the single-prompt formulation: a backbone we initially included, MiniMax-M3, returned empty outputs for most Single-Prompt batches (107 of 156 sentences counted as output failures) but completed the Multi-Step pipeline without failures; we replaced it with Mistral Large 4, and among the six final backbones only DeepSeek-V4-Flash has output failures (3 Single-Prompt sentences).

Where Single-Prompt loses violations. The aggregate gap on Restaurant and SISU has two distinct causes. On Restaurant, one rule (at least one inspection per year) flags 99% of the gold-violating tuples. Every Single-Prompt configuration except Qwen3.8-2.4T-A95B returns `null` for it, so aggregate recall falls to 0.01. Once this rule is excluded, all of them reach 1.00. On SISU, the Single-Prompt mapper identifies the student by the exam registration number instead of the national ID in the two-tuple rules. The resulting DCs flag about 51k tuples instead of 487 (precision below 0.01), although most single-tuple rules are correct. Multi-Step avoids both errors because it links columns and selects predicates before writing the DC.

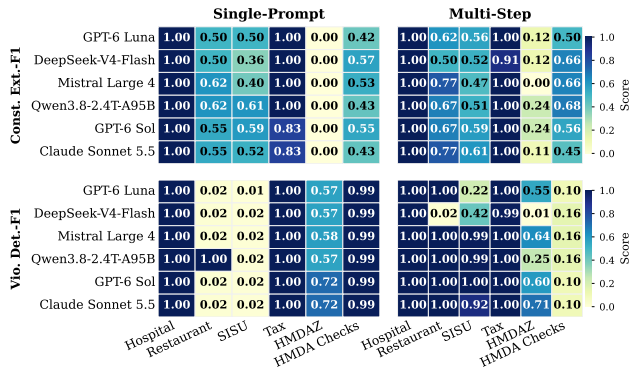


Figure 7: Sentence-level mapping results across models and mappers, measured by constraint extraction F_1 and aggregate violation F_1 on ground-truth sentences only.

Mapper	Model	CE	$CE_{\equiv}$	AV	Rule-F_1	Exec.	Fail.
Single-Prompt	GPT-6 Luna	0.57	0.69	0.60	0.78	0.75	0
	DeepSeek-V4-Flash	0.57	0.66	0.60	0.74	0.70	3
	Mistral Large 4	0.59	0.66	0.60	0.73	0.71	0
	Qwen3.8-2.4T-A95B	0.61	0.70	0.76	**0.85**	**0.82**	0
	GPT-6 Sol	0.59	0.67	0.62	0.76	0.72	0
	Claude Sonnet 5.5	0.55	0.64	0.62	0.75	0.70	0
Multi-Step	GPT-6 Luna	0.63	0.68	0.65	0.75	0.73	0
	DeepSeek-V4-Flash	0.62	0.64	0.43	0.68	0.67	0
	Mistral Large 4	0.65	0.72	**0.80**	0.81	0.80	0
	Qwen3.8-2.4T-A95B	**0.68**	0.70	0.73	0.75	0.73	0
	GPT-6 Sol	0.67	0.71	0.78	0.80	0.79	0
	Claude Sonnet 5.5	0.66	**0.73**	0.79	0.84	0.81	0

Table 3: Mean mapping quality over the six datasets (ground-truth sentences only). CE: exact extraction F_1; $CE_{\equiv}$: equivalence-aware extraction F_1; AV: aggregate violation F_1; Rule-F_1: mean per-rule violation F_1; Exec.: execution equivalence; Fail.: sentences with no valid model output.

HMDA datasets. The two HMDA datasets remain the hardest. Exact extraction stays low (at most 0.24 on HMDAZ and 0.68 on HMDA Checks), mainly for two reasons. Many predicted DCs are logically equivalent to the gold ones but written differently, e.g., `IN` instead of a disjunction, or a label column instead of its code column. Others omit the coverage conditions of a trigger. Equivalence-aware extraction (Table 3) recovers a large part of the first group: on the HMDA datasets it raises the mean extraction F_1 from 0.36 to 0.49 for Multi-Step and from 0.24 to 0.51 for Single-Prompt. On the four original datasets it is almost identical to exact extraction. The aggregate violation ranking is also reversed on HMDA Checks: Single-Prompt reaches 0.99, while Multi-Step stays at 0.10–0.16. This reversal says little about mapping quality. Two date-range edits (S270 and Q022) flag 99% of the gold-violating tuples, which cover 98.6% of the rows, so a DC that flags *every* tuple already scores 0.99. These are the only gold DCs that need arithmetic over dates encoded as integers. Multi-Step returns `null` for both with every backbone, consistent with its precision bias. Single-Prompt writes DCs for them, but most of these DCs flag all rows (e.g., `t1.action_taken_date / 10000 != t1.activity_year`, which never holds as an equality under real division); only GPT-6 Luna, GPT-6 Sol, and Qwen3.8-2.4T-A95B produce an execution-equivalent DC for one of the two. Without these two edits, the ranking flips back: Multi-Step reaches 1.00 with Claude Sonnet 5.5, GPT-6 Luna, and GPT-6 Sol and 0.77 with the other three backbones (mean 0.89), whereas Single-Prompt ranges from 0.18 to 1.00 (mean 0.65). Per-rule metrics, which weigh every rule equally, are far less sensitive to these two edits: on HMDA Checks, the mean per-rule violation F_1 is 0.80 for Multi-Step and 0.86 for Single-Prompt with all edits, and 0.87 and 0.85 without the date edits. On the remaining rules, the two mappers are therefore close, and the aggregate reversal does not reflect the overall quality of the rules.

Default mapper. No single model leads on both metrics of the paper protocol: across the six datasets, Qwen3.8-2.4T-A95B has the best mean extraction (0.68) and Mistral Large 4 the best mean aggregate violation (0.80) with Multi-Step. We therefore use the mean of the two to break the tie and select *GPT-6 Sol* with *Multi-Step* (0.67 and 0.78; mean 0.728, against 0.724 for Mistral Large 4, 0.722 for Claude Sonnet 5.5, and 0.708 for Qwen3.8-2.4T-A95B) as the default sentence-level configuration. Differences among these four are within the variation that a single run per configuration can show, so the choice should be read as a tie among them rather than a clear winner.

ALTI: [R1-O11] Aprofundar a análise comparativa entre os foundation models (a figura "across FMs"): discutir sensibilidade ao modelo, onde e por que modelos divergem, e quais propriedades (tamanho, família, instruction-tuning) correlacionam com desempenho. Hoje a leitura é só "GPT-5 é melhor". Isso também ajuda a rebater a sugestão do revisor de mover para track aplicado, mostrando que há achados de método e não só medição.

6.7 End-to-End Evaluation and Comparison

We now evaluate the full Text-to-Constraints pipeline end-to-end and compare `Norma` against the baselines from Section 6.2. For `Norma`, we use the default components selected in the previous experiments: *Qwen3.8-2.4T-A95B* with *Examples* for constraint classification and *GPT-6 Sol* with *Multi-Step* for mapping. We report two variants of `Norma`: (1) the full pipeline, which combines the first-stage constraint classifier with the Multi-Step mapper (C+MS), and (2) the Multi-Step mapper alone, which skips the initial classification stage and relies on its own rule filter (Multi-Step). As baselines, we include (i) the best result from DC discovery, following the procedure previously described, and (ii) an adaptation of the Text-to-SQL tool XiYan-SQL, as described before. Baseline results on the four original datasets are those of our previous evaluation, which used the same data and gold constraints; the baselines were not run on the HMDA datasets. Table 4 shows the results on all six datasets using the three metrics introduced in Section 6.3: constraint classification, constraint extraction, and violation detection.

`Norma` is the only approach that produces gold constraints on every dataset. Constraint extraction F_1 is 1.00 on Hospital and Tax, 0.67 on Restaurant, 0.53–0.59 on SISU, 0.58–0.59 on HMDA Checks,

Family	Method	Constr. Classification			Constr. Extraction			Viol. Discovery		
		P	R	F_1	P	R	F_1	P	R	F_1
					Hospital					
Norma	C+MS	0.83	1.00	**0.91**	1.00	1.00	**1.00**	1.00	1.00	**1.00**
	Multi-Step	0.71	1.00	0.83	1.00	1.00	**1.00**	1.00	1.00	**1.00**
Profiling	DC Discovery	n/a	n/a	n/a	0.00	0.00	0.00	0.83	1.00	0.91
Text2SQL	XiYan-SQL	0.50	1.00	0.67	0.60	1.00	0.75	0.83	1.00	0.91
					Restaurant					
Norma	C+MS	1.00	0.71	**0.83**	0.80	0.57	**0.67**	1.00	1.00	**1.00**
	Multi-Step	1.00	0.71	**0.83**	0.80	0.57	**0.67**	1.00	1.00	**1.00**
Profiling	DC Discovery	n/a	n/a	n/a	0.00	0.00	0.00	0.77	0.87	0.82
Text2SQL	XiYan-SQL	0.86	0.75	0.80	0.17	0.12	0.14	0.45	0.64	0.52
					SISU					
Norma	C+MS	0.84	0.94	**0.89**	0.59	0.59	**0.59**	0.07	0.99	0.13
	Multi-Step	0.75	0.88	0.81	0.53	0.53	0.53	0.90	0.99	**0.94**
Profiling	DC Discovery	n/a	n/a	n/a	0.00	0.00	0.00	0.00	0.99	0.01
Text2SQL	XiYan-SQL	1.00	0.65	0.79	0.45	0.29	0.36	0.00	1.00	0.01
					Tax					
Norma	C+MS	0.60	1.00	**0.75**	1.00	1.00	**1.00**	0.08	1.00	**0.15**
	Multi-Step	0.60	1.00	**0.75**	1.00	1.00	**1.00**	0.08	1.00	**0.15**
Profiling	DC Discovery	n/a	n/a	n/a	0.00	0.00	0.00	0.00	0.94	0.01
Text2SQL	XiYan-SQL	0.40	1.00	0.57	0.29	0.67	0.40	0.01	1.00	0.02
					HMDAZ					
Norma	C+MS	1.00	0.67	**0.80**	0.25	0.22	0.24	0.47	0.71	0.56
	Multi-Step	1.00	0.67	**0.80**	0.50	0.33	**0.40**	0.58	0.94	**0.72**
Profiling	DC Discovery	–	–	–	–	–	–	–	–	–
Text2SQL	XiYan-SQL	–	–	–	–	–	–	–	–	–
					HMDA Checks					
Norma	C+MS	0.94	0.89	**0.92**	0.65	0.53	0.58	1.00	0.05	**0.10**
	Multi-Step	0.92	0.90	0.91	0.67	0.53	**0.59**	1.00	0.05	**0.10**
Profiling	DC Discovery	–	–	–	–	–	–	–	–	–
Text2SQL	XiYan-SQL	–	–	–	–	–	–	–	–	–

Table 4: End-to-end comparison of Norma modes, Classifier+Multi-Step (C+MS) and Multi-Step, and the baselines across six datasets. Profiling denotes DC discovery from data alone, and Text2SQL denotes XiYan-SQL adapted to generate violation queries from regulatory sentences; their results on the four original datasets come from our previous evaluation, and – marks configurations not run. n/a indicates that the metric is not applicable; bold marks the best F_1 for each metric and dataset.

and 0.24–0.40 on HMDAZ, against at most 0.75 for XiYan-SQL and zero for DC discovery on the original datasets. Classification is also strong, with F_1 between 0.75 and 0.92 for C+MS. The pipeline therefore extends to the two HMDA datasets, with up to 135 columns and 1M rows, against at most 56 columns and 128k rows in the original ones. On HMDA Checks, the 92 constraint edits are classified with F_1 0.92 and more than half of them are extracted exactly.

The classifier mainly improves precision. Compared with *Multi-Step* alone, *C+MS* raises classification precision on Hospital (0.83 vs. 0.71), SISU (0.84 vs. 0.75), and HMDA Checks (0.94 vs. 0.92), and it reduces the non-constraint sentences that reach a DC (3 vs. 6 on Hospital, 3 vs. 5 on SISU, 5 vs. 7 on HMDA Checks). With the current backbones, however, the gain is smaller than in our previous evaluation. The Multi-Step rule filter already rejects most non-constraint sentences, and on Restaurant, Tax, and HMDAZ both variants make the same classification decisions. On HMDAZ, C+MS is worse on extraction and violations (0.24 vs. 0.40 and 0.56 vs. 0.72). The two variants map five common sentences and differ on one each: the classifier drops the balloon-payment limitation, which Multi-Step maps exactly, and keeps the points-and-fees trigger, which the Multi-Step filter rejects. The two runs also write different DCs for the APR trigger (per-rule violation F_1 0.17 vs. 0.35). Part of the gap is therefore run-to-run variation of a single execution.

The violation results expose two failure modes that are specific to the end-to-end setting, where non-constraint sentences can also reach the mapper. On Tax, every gold constraint is extracted exactly (F_1 1.00), yet aggregate violation F_1 falls to 0.15. Four of the 120 non-constraint sentences are accepted by the classifier and the rule filter and mapped to DCs. These sentences were written to resemble constraints (e.g., "The sample data shows exemption values ranging from $0 to $6,600, all falling well within respective salary figures"), and the resulting DCs flag 5.6k tuples against 468 gold violations. On SISU, the C+MS run maps one gold sentence with an additional two-tuple rule that identifies students by exam registration instead of national ID, flagging 6.9k tuples (aggregate F_1 0.13 vs. 0.94 for Multi-Step, whose DC for the same sentence uses the national ID). Both cases show that a single broad DC dominates the tuple-level aggregate. On HMDA Checks, the aggregate (0.10) again reflects the two date-range edits discussed in Section 6.6. The per-rule view is more informative there: 83% of the gold rules with violations are reproduced exactly (execution equivalence 0.83 for both variants), and the mean per-rule violation F_1 is 0.80–0.83.

The baselines are competitive only on narrower parts of the experiments, that is, on specific aspects of the evaluation. The profiling baseline can recover meaningful violation signals when the data distribution strongly reflects the target rules, but it does not recover the gold constraint set itself, with constraint-extraction F_1 equal to zero on all datasets. XiYan-SQL is competitive on Hospital, but its behavior is much less reliable on the remaining datasets. On SISU and Tax in particular, it achieves very high recall for violations but near-zero precision, indicating that it returns queries pointing to overly broad violation sets.

Finally, the dataset characteristics matter when interpreting the aggregate violation results. In four of the six datasets, a single gold rule flags most of the gold-violating tuples (96% on Hospital, 99% on Restaurant, 78% on Tax, and 99% on HMDA Checks). The aggregate then mostly measures whether that rule is recovered. Methods that return broad violation sets can still overlap strongly with the gold violations, as with the profiling baseline on Hospital and Restaurant. Conversely, a correct pipeline can be penalized by one broad false positive, as on Tax. We therefore recommend reading the aggregate together with the per-rule metrics of Section 6.3. Overall, Norma is the only approach that remains strong on classification and extraction across all six datasets. The violation metric remains sensitive

to individual broad DCs.

ALTI: [R1-O9] Responder à usabilidade do output: com uma constraint por sentença, o conjunto retornado pode ser grande. Discutir como o usuário tria isso, p.ex. ranqueamento por nº de violações/severidade, rastreabilidade ao trecho de origem como filtro, e a revisão humana opcional já prevista no pipeline. Reportar o tamanho típico do conjunto retornado por dataset para dimensionar o esforço.

Costs. The full sentence-level Norma pipeline is the most expensive variant because it requires multiple model calls per sentence. In this evaluation, all experiments together cost US$83 through OpenRouter (E1 US$5.8, E2 US$52.7, end-to-end US$24.5). With GPT-6 Sol, an end-to-end run costs between US$0.3 and US$2.3 on the original datasets and between US$3.4 (C+MS) and US$9.9 (Multi-Step) on HMDA Checks (158 sentences, 1M rows). The classifier makes the pipeline 1.6 to 5 times cheaper, because rejected sentences never reach the Multi-Step mapper. Low-cost backbones such as GPT-6 Luna and DeepSeek-V4-Flash cost a few cents per dataset. Cost scales linearly with the number of sentences, making the approach tractable in practice. By contrast, data profiling baselines face combinatorial search over the schema, and for some datasets we had to restrict the columns considered.

7 Related Work

Data Profiling. The closest line of work to Norma is *data profiling*, which aims to discover constraints directly from data instances [2, 9, 15, 21, 27, 44, 45, 48]. This includes formalisms such as functional dependencies, conditional dependencies, and denial constraints (DCs). DCs are particularly relevant in our setting because they subsume keys, FDs, and ordering constraints [15], making them expressive enough to capture many cross-tuple rules in regulatory text. Approximate discovery tolerates violations in dirty data [40, 46], user-specific criteria help experts rank discovered rules [20], and LLMs now propose candidate rules from schemas and samples [8, 23, 32]. Still, all these methods are driven by the observed data. They reveal what the data seems to satisfy, but not whether that pattern is required by policy or regulation. As a result, profiling cannot recover constraints that are already violated in the released data, and discovered DCs often include many spurious constraints [39]. Our goal is to recover constraints from normative text and then use them to validate the data.

ALTI: [R3-O1, R1-O1] Completar o related work: (i) citar e discutir descoberta APROXIMADA de DCs (VLDB 2019) e trabalho de guiar especialistas na definição de regras, alinhando com a correção feita na intro; (ii) adicionar REEs (regras com ML-predicates) como linha relacionada e contrastar o espaço de predicados deles com o nosso vocabulário controlado.

Data Cleaning. Data cleaning systems use constraints, statistical signals, or both to detect and repair errors [1, 16, 38, 42, 49, 52], including rules with ML predicates (REEs) [6] and LLM-based detectors [43, 62]. These systems complement our setting. Constraint-based methods are interpretable and auditable, but they assume that the relevant rules are already available. Data-driven methods reduce the need for manual rule specification, but without an external source of truth, they can struggle to distinguish rare valid values from genuine errors [42]. Our work addresses this missing step of obtaining such rules from regulatory documents. We use DCs over a controlled, schema-derived predicate space as the output formalism, so every constraint is executable and auditable, and, as prior work has shown, DCs can be checked efficiently at scale [29, 37, 47].

Regulatory NLP. A separate line of work uses NLP to analyze regulations and compliance-related documents, for example, by extracting rights and obligations or by checking whether requirement statements comply with legal or policy text [10, 13, 18, 25, 30, 41]. LLMs now extract structured rules from regulations [24, 26, 28] and translate financial regulations into compliance code [33], although they still struggle to apply long sets of interdependent rules [58, 60]. These approaches remain text-centered or target a fixed application model. Our problem starts from similar documents, but we need to connect regulatory statements to a relational schema and translate them into constraints whose violations can be checked in the data.

Text-to-SQL. Text-to-SQL is also related, since it maps natural-language input to a SQL query over a database schema [4, 22, 31, 36, 50, 51, 53, 54, 56, 61]. The difference lies in how the generated logic is used, as Text-to-SQL produces a query that answers a user question on the current database instance, whereas Norma aims at declarative invariants that should hold for all valid instances of the data. In addition to mapping text to schema elements, as in Text-to-SQL, it must also identify scope conditions and define violation semantics. Closer in output, LLMs generate OCL constraints from specifications of UML models [3, 5], and Doc2DB systems verify extracted relations against integrity constraints [63]; in both cases, the target schema is designed together with the constraints, whereas Norma grounds regulatory text in an independently published dataset.

8 Conclusion

A constraint inferred from data describes what the data happens to satisfy, whereas a constraint contained in governing text describes what the data is required to satisfy. This distinction has received limited attention in database research, even though many published datasets are accompanied by documents that define data quality criteria at some level.

In this paper, we show that the gap between data and normative text can be addressed automatically. Our results indicate that lightweight classification is sufficient to detect most checkable requirements, and that a structured multi-step pipeline grounded in both text and schema can translate them into executable constraints while avoiding hallucinated predicates and literals.

This enables a new form of validation: Norma can reveal compliance failures even when they do not manifest as discoverable pattern in the data. In such settings, the normative document becomes part of the data specification, and keeping that specification aligned with evolving text emerges a concrete challenge for data management.

ALTI: [R2-O6.1] Reconhecer como limitação que os datasets são pequenos em nº de sentenças e que só SISU se aproxima do cenário real. Adicionar discussão de escalabilidade para documentos legais típicos (centenas de sentenças, cláusulas aninhadas): custo linear por sentença já é argumento a favor; discutir segmentação de cláusulas aninhadas e o modo document-level como rota, e apontar isso como trabalho futuro.

ALTI: [R1-O3] Responder à reusabilidade sob dados dinâmicos: como a constraint é extraída do TEXTO (não dos dados), ela permanece válida enquanto a norma não muda, e pode ser re-executada sobre novas instâncias/atualizações sem re-descoberta. Conectar com a observação final sobre manter a especificação alinhada ao texto em evolução. Distinguir "dados mudam" (constraint estável) de "norma muda" (re-extração).

References

[1] Ziawasch Abedjan, Xu Chu, Dong Deng, Raul Castro Fernandez, Ihab F. Ilyas, Mourad Ouzzani, Paolo Papotti, Michael Stonebraker, and Nan Tang. 2016. Detecting data errors: where are we and what needs to be done? *PVLDB* 9, 12 (Aug. 2016), 993–1004. doi:10.14778/2994509.2994518

[2] Ziawasch Abedjan, Lukasz Golab, and Felix Naumann. 2015. Profiling relational data: a survey. *The VLDB Journal* 24, 4 (Aug. 2015), 557–581. doi:10.1007/s00778-015-0389-y

[3] Seif Abukhalaf, Mohammad Hamdaqa, and Foutse Khomh. 2023. On Codex Prompt Engineering for OCL Generation: An Empirical Study. In *Proceedings of the 20th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 148–157. doi:10.1109/MSR59073.2023.00033

[4] Tarfah Alrashed, Madhup Sukoon, David R. Karger, and Natasha Noy. 2026. Developing and Benchmarking Verification Algorithms to Improve Text-to-SQL Generation. *Proc. VLDB Endow.* 19, 11 (2026), 3732–3744. doi:10.14778/3836663.3836721

[5] Hamza Attarwala, Moataz Chouchen, Mohammad Hamdaqa, and Omar Alam. 2026. Why Do LLMs Fail at OCL Generation? A Graph Reasoning Perspective. In *Proceedings of the 20th International Symposium on Empirical Software Engineering and Measurement (ESEM) (LIPIcs, Vol. 394)*. doi:10.4230/LIPIcs.ESEM.2026.48

[6] Xianchun Bao, Zian Bao, Binbin Bie, QingSong Duan, Wenfei Fan, Hui Lei, Daji Li, Wei Lin, Peng Liu, Zhicong Lv, Mingliang Ouyang, Shuai Tang, Yaoshu Wang, Qiyuan Wei, Min Xie, Jing Zhang, Xin Zhang, Runxiao Zhao, and Shuping Zhou. 2024. Rock: Cleaning Data by Embedding ML in Logic Rules. In *Companion of the 2024 International Conference on Management of Data (SIGMOD-Companion)*. 106–119. doi:10.1145/3626246.3653372

[7] Carlo Batini, Cinzia Cappiello, Chiara Francalanci, and Andrea Maurino. 2009. Methodologies for data quality assessment and improvement. *ACM Comput. Surv.* 41, 3, Article 16 (July 2009), 52 pages. doi:10.1145/1541880.1541883

[8] Fabian Biester, Mohamed Abdelaal, and Daniel Del Gaudio. 2024. LLM-Clean: Context-Aware Tabular Data Cleaning via LLM-Generated OFDs. *CoRR* abs/2404.18681 (2024).

[9] Tobias Bleifuß, Sebastian Kruse, and Felix Naumann. 2017. Efficient Denial Constraint Discovery with Hydra. *PVLDB* 11, 3 (2017), 311–323.

[10] Travis D. Breaux, Matthew W. Vail, and Annie I. Antón. 2006. Towards Regulatory Compliance: Extracting Rights and Obligations to Align Requirements with Regulations. In *Proceedings of the 14th IEEE International Requirements Engineering Conference (RE)*. IEEE Computer Society, 46–55.

[11] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 159, 25 pages.

[12] Ismael Caballero, Fernando Gualo, Moisés Rodríguez, and Mario Piattini. 2022. BR4DQ: A methodology for grouping business rules for data quality evaluation. *Information Systems* 109 (2022), 102058. doi:10.1016/j.is.2022.102058

[13] Orlando Amaral Cejas, Muhammad Ilyas Azeem, Sallam Abualhaija, and Lionel C. Briand. 2023. NLP-Based Automated Compliance Checking of Data Processing Agreements Against GDPR. *IEEE Trans. Softw. Eng.* 49, 9 (Sept. 2023), 4282–4303. doi:10.1109/TSE.2023.3288901

[14] Adriane Chapman, Elena Simperl, Laura Koesten, George Konstantinidis, Luis-Daniel Ibáñez, Emilia Kacprzak, and Paul Groth. 2020. Dataset search: a survey. *The VLDB Journal* 29, 1 (2020), 251–272. doi:10.1007/s00778-019-00564-x

[15] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Discovering denial constraints. *PVLDB* 6, 13 (Aug. 2013), 1498–1509. doi:10.14778/2536258.2536262

[16] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Holistic data cleaning: Putting violations into context. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. 458–469. doi:10.1109/ICDE.2013.6544847

[17] Kevin L. Cope. 2023. Measuring law's normative force. *Journal of Empirical Legal Studies* 20, 4 (2023), 1005–1044. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/jels.12364 doi:10.1111/jels.12364

[18] Souvick Das, Novarun Deb, Nabendu Chaki, and Agostino Cortesi. 2025. A Multi-Agent RAG Framework for Regulatory Compliance Checking of Software Requirements. *ACM Trans. Softw. Eng. Methodol.* (Dec. 2025). doi:10.1145/3785472 Just Accepted.

[19] Ruanitto Docini, Eduardo C. de Almeida, and Luiz S. Oliveira. 2025. The Wrecking SQL Incremental Validation Methodology. In *Database and Expert Systems Applications - 36th International Conference, DEXA 2025, Bangkok, Thailand, August 25-27, 2025, Proceedings, Part II (Lecture Notes in Computer Science)*. Springer, 376–382. doi:10.1007/978-3-032-02088-8_30

[20] Wenfei Fan, Ziyan Han, Yaoshu Wang, and Min Xie. 2023. Discovering Top-k Rules using Subjective and Objective Criteria. *Proc. ACM Manag. Data* 1, 1 (2023), 1–29. doi:10.1145/3588924

[21] Wenfei Fan, Jianhua Li, Shuai Ma, and Nan Tang. 2009. Discovering Conditional Functional Dependencies. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 1231–1234.

[22] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. VLDB Endow.* 17, 5 (Jan. 2024), 1132–1145. doi:10.14778/3641204.3641221

[23] Anna-Christina Glock, Thomas Hütter, Johannes Fürnkranz, Wolfram Wöß, Christine Dominka-Kiss, and Lisa Ehrlinger. 2026. Data Quality Rule Generation with LLMs. *CoRR* abs/2609.06053 (2026).

[24] Keerat Guliani, Deepkamal Gill, David Landsman, Nima Eshraghi, Krishna Kumar, and Lovedeep Gondara. 2026. De Jure: Iterative LLM Self-Refinement for Structured Extraction of Regulatory Rules. *CoRR* abs/2604.02276 (2026).

[25] Shabnam Hassani, Mehrdad Sabetzadeh, Daniel Amyot, and Jain Liao. 2024. Rethinking Legal Compliance Automation: Opportunities with Large Language Models. In *Proceedings of the 32nd IEEE International Requirements Engineering Conference (RE)*. 432–440. doi:10.1109/RE59067.2024.00051

[26] Hansi Hettiarachchi, Amna Dridi, Mohamed Medhat Gaber, Pouyan Parsafard, Nicoleta Bocaneala, Katja Breitenfelder, Gonçal Costa, Maria Hedblom, Mihaela Juganaru-Mathieu, Thamer Mecharnia, Sumee Park, He Tan, Abdel-Rahman H. Tawil, and Edlira Vakaj. 2025. CODE-ACCORD: A Corpus of building regulatory data for rule generation towards automatic compliance checking. *Scientific Data* 12, 1 (2025), 170.

[27] Ykä Huhtala, Juha Kärkkäinen, Pasi Porkka, and Hannu Toivonen. 1998. TANE: An Efficient Algorithm for Discovering Functional and Approximate Dependencies. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 392–401.

[28] Zarina Kabzhan, Alexandr Shakhnovich, Sergey Gorshkov, Yussuf Yemenov, Fedor Gorshkov, and Nazym Shogelova. 2025. Semantic and ontology-based analysis of regulatory documents for construction industry digitalization. *Frontiers in Built Environment* 11 (2025), 1575913. doi:10.3389/fbuil.2025.1575913

[29] Youri Kaminsky, Eduardo H. M. Pena, and Felix Naumann. 2025. Incremental Detection of Denial Constraint Violations. *PVLDB* 18, 4 (May 2025), 1000–1012. doi:10.14778/3717755.3717761

[30] Nadzeya Kiyavitskaya, Nicola Zeni, Travis D. Breaux, Annie I. Antón, James R. Cordy, Luisa Mich, and John Mylopoulos. 2008. Automating the Extraction of Rights and Obligations for Regulatory Compliance. In *Conceptual Modeling - ER 2008, 27th International Conference on Conceptual Modeling, Barcelona, Spain, October 20-24, 2008. Proceedings (Lecture Notes in Computer Science, Vol. 5231)*. Springer, 154–168.

[31] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. RESDSQL: decoupling schema linking and skeleton parsing for text-to-SQL. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'23/IAAI'23/EAAI'23)*. AAAI Press, Article 1466, 9 pages. doi:10.1609/aaai.v37i11.26535

[32] Qing Li, Yanyan Shen, Qibin Zheng, Yi Liu, and Xingchun Diao. 2026. LCATS: LLM-Guided Constraint-Aware Tabular Data Synthesis. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD)*. 2698–2707. doi:10.1145/3770855.3818181

[33] Siyuan Li, Jian Chen, Rui Yao, Xuming Hu, Peilin Zhou, Weihua Qiu, Simin Zhang, Chucheng Dong, Zhiyao Li, Qipeng Xie, and Zixuan Yuan. 2025. Compliance-to-Code: Enhancing Financial Compliance Checking via Code Generation. *CoRR* abs/2505.19804 (2025).

[34] Xi Victoria Lin, Richard Socher, and Caiming Xiong. 2020. Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 4870–4888. doi:10.18653/v1/2020.findings-emnlp.438

[35] Chang Liu, Arif Usta, Jian Zhao, and Semih Salihoglu. 2023. Governor: Turning Open Government Data Portals into Interactive Databases. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 415, 16 pages. doi:10.1145/3544548.3580868

[36] Yifu Liu, Yin Zhu, Yingqi Gao, Zhiling Luo, Xiaoxia Li, Xiaorong Shi, Yuntao Hong, Jinyang Gao, Yu Li, Bolin Ding, and Jingren Zhou. 2026. XiYan-SQL: A Novel Multi-Generator Framework for Text-to-SQL. *IEEE Transactions on Knowledge and Data Engineering* (2026), 1–14. doi:10.1109/TKDE.2026.3657851

[37] Zifan Liu, Shaleen Deep, Anna Fariha, Fotis Psallidas, Ashish Tiwari, and Avrilia Floratou. 2024. Rapidash: Efficient Detection of Constraint Violations. *Proc. VLDB Endow.* 17, 8 (April 2024), 2009–2021. doi:10.14778/3659437.3659454

[38] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A Configuration-Free Error Detection System. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 865–882. doi:10.1145/3299869.3324956

[39] Albert Martin, Eduardo C. de Almeida, Oscar Romero, and Anna Queralt. 2025. How and Why False Denial Constraints are Discovered. *PVLDB* 18, 10 (2025), 3477–3489.

[40] Albert Martin, Eduardo C. de Almeida, Oscar Romero, and Anna Queralt. 2026. Discovering Approximate Denial Constraints in Large Databases. *Proc. VLDB Endow.* 19, 9 (2026), 2086–2098. doi:10.14778/3819518.3819536

[41] Arsalan Masoudifard, Mohammad Mowlavi Sorond, Moein Madadi, Mohammad Sabokrou, and Elahe Habibi. 2024. Leveraging Graph-RAG and Prompt Engineering to Enhance LLM-Based Automated Requirement Traceability and Compliance Checks. *CoRR* abs/2412.08593 (2024).

[42] Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, and Jianwei Yin. 2024. Automatic Data Repair: Are We Ready to Deploy? *PVLDB* 17, 10 (June 2024), 2617–2630. doi:10.14778/3675034.3675051

[43] Wei Ni, Kaihang Zhang, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Yaoshu Wang, and Jianwei Yin. 2025. ZeroED: Hybrid Zero-Shot Error Detection Through Large Language Model Reasoning. In *Proceedings of the 41st IEEE International Conference on Data Engineering (ICDE)*. 3126–3139. doi:10.1109/ICDE65448.2025.00234

[44] Thorsten Papenbrock and Felix Naumann. 2016. A Hybrid Approach to Functional Dependency Discovery. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 821–833.

[45] Eduardo Pena, Fabio Porto, and Felix Naumann. 2024. Discovering Denial Constraints in Dynamic Datasets. 3546–3558. doi:10.1109/ICDE60146.2024.00273

[46] Eduardo H. M. Pena, Eduardo C. de Almeida, and Felix Naumann. 2019. Discovery of Approximate (and Exact) Denial Constraints. *PVLDB* 13, 3 (2019), 266–278.

[47] Eduardo H. M. Pena, Eduardo C. de Almeida, and Felix Naumann. 2021. Fast detection of denial constraint violations. *PVLDB* 15, 4 (Dec. 2021), 859–871. doi:10.14778/3503585.3503595

[48] Eduardo H. M. Pena, Fabio Porto, and Felix Naumann. 2022. Fast Algorithms for Denial Constraint Discovery. *PVLDB* 16, 4 (2022), 684–696.

[49] Jinfeng Peng, Derong Shen, Nan Tang, Tieying Liu, Yue Kou, Tiezheng Nie, Hang Cui, and Ge Yu. 2022. Self-Supervised and Interpretable Data Cleaning with Sequence Generative Adversarial Networks. *Proc. VLDB Endow.* 16, 3 (Nov. 2022), 433–446. doi:10.14778/3570690.3570694

[50] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: decomposed in-context learning of text-to-SQL with self-correction. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 1577, 10 pages.

[51] Ricardo Trainotti Rabonato and Lilian Berton. 2026. The Impact of Language and Portuguese Intralinguistic Variation on NL2SQL. In *Anais do XVII Simpósio Brasileiro de Tecnologia da Informação e da Linguagem Humana (STIL)*. 317–330. doi:10.5753/stil.2026.26611

[52] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: holistic data repairs with probabilistic inference. *PVLDB* 10, 11 (Aug. 2017), 1190–1201. doi:10.14778/3137628.3137631

[53] Tonghui Ren, Chen Ke, Yuankai Fan, Yinan Jing, Zhenying He, Kai Zhang, and X. Sean Wang. 2025. The Power of Constraints in Natural Language to SQL Translation. *Proc. VLDB Endow.* 18, 7 (March 2025), 2097–2111. doi:10.14778/3734839.3734847

[54] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 9895–9901. doi:10.18653/v1/2021.emnlp-main.779

[55] Flavia Serra, Verónika Peralta, Adriana Marotta, and Patrick Marcel. 2024. Use of Context in Data Quality Management: A Systematic Literature Review. *J. Data and Information Quality* 16, 3, Article 19 (Oct. 2024), 41 pages. doi:10.1145/3672082

[56] Liang Shi, Zhengju Tang, Nan Zhang, Xiaotong Zhang, and Zhi Yang. 2025. A Survey on Employing Large Language Models for Text-to-SQL Tasks. *ACM Comput. Surv.* 58, 2, Article 54 (Sept. 2025), 37 pages. doi:10.1145/3737873

[57] Shaoxu Song, Fei Gao, Ruihong Huang, and Chaokun Wang. 2020. Data Dependencies over Big Data: A Family Tree. *IEEE Transactions on Knowledge and Data Engineering* PP (12 2020), 1–1. doi:10.1109/TKDE.2020.3046443

[58] Utkarsh Soni, Syed Shariyar Murtaza, Yifan Nie, Sachin Chandrasekhar, and Eugene Wen. 2026. Tasks over Application Manuals: Revealing Gaps in Long-Horizon Procedural Reasoning for Language Models. In *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM)*. doi:10.1145/3799682.3841039

[59] Xiaofei Sun, Xiaoya Li, Jiwei Li, Fei Wu, Shangwei Guo, Tianwei Zhang, and Guoyin Wang. 2023. Text Classification via Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 8990–9005. doi:10.18653/v1/2023.findings-emnlp.603

[60] Bohan Yu, Pengfei Cao, Chen Han, Chenxi Zhou, Zhiheng Zhang, Zhiyang Xie, Wenhao Teng, Xiangwen Liao, Jun Zhao, and Kang Liu. 2026. Beyond Factual Knowledge: Benchmarking and Learning Step-Level Procedural Rule Reasoning in Large Language Models. *CoRR* abs/2608.22753 (2026).

[61] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Brussels, Belgium, 3911–3921.

[62] Shuo Zhang, Zezhou Huang, and Eugene Wu. 2024. Data Cleaning Using Large Language Models. *CoRR* abs/2410.15547 (2024).

[63] Zhengxuan Zhang, Zhuowen Liang, Jiazhuo Chen, Haixun Wang, and Nan Tang. 2026. Document-to-Database: Extraction Meets Relational Semantics. *Proc. VLDB Endow.* 19, 9 (2026), 2522–2532. doi:10.14778/3819518.3819568

[64] Xiaoyan Zhao, Yang Deng, Min Yang, Lingzhi Wang, Rui Zhang, Hong Cheng, Wai Lam, Ying Shen, and Ruifeng Xu. 2024. A Comprehensive Survey on Relation Extraction: Recent Advances and New Frontiers. *ACM Comput. Surv.* 56, 11, Article 293 (July 2024), 39 pages. doi:10.1145/3674501

[65] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic Evaluation for Text-to-SQL with Distilled Test Suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 396–411. doi:10.18653/v1/2020.emnlp-main.29